In [1]:
```
import sys
print(sys.version)

import pycaret
print(pycaret.__version__)
```

```
3.10.20 | packaged by conda-forge | (main, Jun 11 2026, 03:31:56) [GCC 14.3.0]
3.3.2
```

In [2]:
```
!pip install pandas s3fs
```

```
Requirement already satisfied: pandas in /opt/conda/lib/python3.12/site-packages (2.3.3)
Requirement already satisfied: s3fs in /opt/conda/lib/python3.12/site-packages (2026.6.0)
Requirement already satisfied: numpy>=1.26.0 in /opt/conda/lib/python3.12/site-packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/conda/lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.12/site-packages (from pandas) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /opt/conda/lib/python3.12/site-packages (from pandas) (2026.3)
Requirement already satisfied: aiobotocore<4.0.0,>=2.19.0 in /opt/conda/lib/python3.12/site-packages (from s3fs) (3.8.0)
Requirement already satisfied: fsspec<2026.6.1,>=2026.6.0 in /opt/conda/lib/python3.12/site-packages (from s3fs) (2026.6.0)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1,>=3.9.0 in /opt/conda/lib/python3.12/site-packages (from s3fs) (3.14.2)
Requirement already satisfied: aioitertools<1.0.0,>=0.5.1 in /opt/conda/lib/python3.12/site-packages (from aiobotocore<4.0.0,>=2.1
9.0->s3fs) (0.13.0)
Requirement already satisfied: botocore<1.43.47,>=1.43.3 in /opt/conda/lib/python3.12/site-packages (from aiobotocore<4.0.0,>=2.19.
0->s3fs) (1.43.46)
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /opt/conda/lib/python3.12/site-packages (from aiobotocore<4.0.0,>=2.19.0->
s3fs) (1.1.0)
Requirement already satisfied: multidict<7.0.0,>=6.0.0 in /opt/conda/lib/python3.12/site-packages (from aiobotocore<4.0.0,>=2.19.0-
>s3fs) (6.7.1)
Requirement already satisfied: wrapt<3.0.0,>=1.10.10 in /opt/conda/lib/python3.12/site-packages (from aiobotocore<4.0.0,>=2.19.0->s
3fs) (1.17.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /opt/conda/lib/python3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1,
>=3.9.0->s3fs) (2.7.1)
Requirement already satisfied: aiosignal>=1.4.0 in /opt/conda/lib/python3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1,>=3.9.0
->s3fs) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /opt/conda/lib/python3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1,>=3.9.0->s
3fs) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /opt/conda/lib/python3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1,>=3.9.
0->s3fs) (1.8.0)
Requirement already satisfied: propcache>=0.2.0 in /opt/conda/lib/python3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1,>=3.9.0
->s3fs) (0.5.2)
Requirement already satisfied: typing_extensions>=4.4 in /opt/conda/lib/python3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1,>
=3.9.0->s3fs) (4.16.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /opt/conda/lib/python3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1,>=3.9.
0->s3fs) (1.24.5)
Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in /opt/conda/lib/python3.12/site-packages (from botocore<1.43.47,>=1.43.
3->aiobotocore<4.0.0,>=2.19.0->s3fs) (1.26.20)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: idna>=2.0 in /opt/conda/lib/python3.12/site-packages (from yarl<2.0,>=1.17.0->aiohttp!=4.0.0a0,!=4.
0.0a1,>=3.9.0->s3fs) (3.18)
```

In [3]:

```python
import awswrangler as wr

s3_path = 's3://databogota/bogota_ivc_thesis_ready.csv'

# Leer el archivo CSV
df = wr.s3.read_csv(path=s3_path)

df.head()
```

Out[3]:

	CD_LC_CM	NMB_LC_CM	TP_LC_CM	AREA	LATITUD	LONGITUD	Shape_Leng	Shape_Area	TP27_PERSO	TVIVIENDA	...	TP19_EE_E
0	4.0	SAN CRISTOBAL	Localidad	8555.858874	4.585633	-74.081623	0.004176	6.972582e-07	13	4	...	
1	4.0	SAN CRISTOBAL	Localidad	17684.725840	4.584788	-74.080810	0.005631	1.441212e-06	442	141	...	
2	4.0	SAN CRISTOBAL	Localidad	55950.125530	4.582480	-74.081045	0.011082	4.559626e-06	787	373	...	
3	4.0	SAN CRISTOBAL	Localidad	34563.126910	4.582606	-74.083257	0.008378	2.816703e-06	2511	906	...	
4	4.0	SAN CRISTOBAL	Localidad	5707.732035	4.582665	-74.079530	0.002777	4.651488e-07	224	57	...	

5 rows × 62 columns

In [4]:
```
df.describe()
```

Out[4]:

	CD_LC_CM	AREA	LATITUD	LONGITUD	Shape_Leng	Shape_Area	TP27_PERSO	TVIVIENDA	TP16_HOG	DEN
count	43408.000000	4.340800e+04	43408.000000	43408.000000	43408.000000	4.340800e+04	43408.000000	43408.000000	43408.000000	43408.(
mean	10.047226	7.333356e+03	4.625294	-74.116708	0.002768	5.976834e-07	164.414670	58.001590	57.832335	0.(
std	5.402681	5.876433e+04	0.073093	0.042491	0.002859	4.789465e-06	267.909685	104.436705	95.488752	0.(
min	1.000000	3.224927e+01	3.890001	-74.364400	0.000211	2.628032e-09	0.000000	0.000000	0.000000	0.(
25%	6.000000	1.478788e+03	4.569291	-74.148485	0.001570	1.205110e-07	51.000000	17.000000	18.000000	0.(
50%	10.000000	2.623964e+03	4.617878	-74.113940	0.002168	2.138725e-07	113.000000	37.000000	39.000000	0.(
75%	14.000000	5.154551e+03	4.684844	-74.088905	0.003000	4.201348e-07	191.000000	63.000000	66.000000	0.(
max	19.000000	9.064324e+06	4.826050	-74.008754	0.144168	7.388130e-04	8330.000000	4618.000000	2863.000000	0.5

8 rows × 58 columns

In [5]:
```
df.columns.tolist()
```

Out[5]:
```
['CD_LC_CM',
 'NMB_LC_CM',
 'TP_LC_CM',
 'AREA',
 'LATITUD',
 'LONGITUD',
 'Shape_Leng',
 'Shape_Area',
 'TP27_PERSO',
 'TVIVIENDA',
 'TP16_HOG',
 'DENSIDAD',
 'TP34_1_EDA',
 'TP34_8_EDA',
 'TP34_9_EDA',
 'TP51_13_ED',
 'TP51POSTGR',
 'TP51PRIMAR',
 'TP51SECUND',
 'TP51SUPERI',
 'TP51_99_ED',
 'TP14_6_TIP',
 'TP15_4_OCU',
 'TP9_3_10_N',
 'TP9_3_6_NO',
 'TP14_3_TIP',
 'TP15_1_OCU',
 'TP15_2_OCU',
 'TP15_3_OCU',
 'TP9_3_8_NO',
 'TP9_3_9_NO',
 'TP9_2_4_MI',
 'TP9_3_4_NO',
 'TP19_EE_2',
 'TP19_ACU_2',
 'TP19_ALC_2',
 'TP19_GAS_2',
 'TP19_RECB2',
 'TP19_INTE2',
 'TP19_GAS_9',
 'TP19_INTE9',
 'TP19_EE_1',
 'TP19_ACU_1',
 'TP19_ALC_1',
 'TP19_GAS_1',
 'TP19_RECB1',
 'TP19_INTE1',
```

```
'TP19_EE_E1',
'TP19_EE_E2',
'TP19_EE_E3',
'TP19_EE_E4',
'TP19_EE_E5',
'TP19_EE_E6',
'TP19_EE_E9',
'TP9_1_USO',
'TP9_2_USO',
'TP9_3_USO',
'ppt_mean_2000_2018',
'ivc_formula',
'ivc_n',
'categoria_ivc',
'perfil_vulnerabilidad']
```

In [6]:
```
df = df.drop(columns=['ivc_formula', 'categoria_ivc', 'perfil_vulnerabilidad'])
```

In [7]:
```
!pip install pycaret
```

```
Collecting pycaret
  Using cached pycaret-3.3.2-py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: ipython>=5.5.0 in /opt/conda/lib/python3.12/site-packages (from pycaret) (8.37.0)
Requirement already satisfied: ipywidgets>=7.6.5 in /opt/conda/lib/python3.12/site-packages (from pycaret) (8.1.8)
Requirement already satisfied: tqdm>=4.62.0 in /opt/conda/lib/python3.12/site-packages (from pycaret) (4.69.0)
Requirement already satisfied: numpy<1.27,>=1.21 in /opt/conda/lib/python3.12/site-packages (from pycaret) (1.26.4)
Collecting pandas<2.2.0 (from pycaret)
  Using cached pandas-2.1.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (18 kB)
Requirement already satisfied: jinja2>=3 in /opt/conda/lib/python3.12/site-packages (from pycaret) (3.1.6)
Collecting scipy<=1.11.4,>=1.6.1 (from pycaret)
  Using cached scipy-1.11.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (60 kB)
Collecting joblib<1.4,>=1.2.0 (from pycaret)
  Using cached joblib-1.3.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: scikit-learn>1.4.0 in /opt/conda/lib/python3.12/site-packages (from pycaret) (1.7.2)
Collecting pyod>=1.1.3 (from pycaret)
  Using cached pyod-3.6.4-py3-none-any.whl.metadata (59 kB)
Collecting imbalanced-learn>=0.12.0 (from pycaret)
  Using cached imbalanced_learn-0.14.2-py3-none-any.whl.metadata (8.9 kB)
Collecting category-encoders>=2.4.0 (from pycaret)
  Using cached category_encoders-2.10.0-py3-none-any.whl.metadata (8.0 kB)
Requirement already satisfied: lightgbm>=3.0.0 in /opt/conda/lib/python3.12/site-packages (from pycaret) (4.6.0)
Requirement already satisfied: numba>=0.55.0 in /opt/conda/lib/python3.12/site-packages (from pycaret) (0.66.0)
Requirement already satisfied: requests>=2.27.1 in /opt/conda/lib/python3.12/site-packages (from pycaret) (2.34.2)
Requirement already satisfied: psutil>=5.9.0 in /opt/conda/lib/python3.12/site-packages (from pycaret) (5.9.8)
Requirement already satisfied: markupsafe>=2.0.1 in /opt/conda/lib/python3.12/site-packages (from pycaret) (3.0.3)
Requirement already satisfied: importlib-metadata>=4.12.0 in /opt/conda/lib/python3.12/site-packages (from pycaret) (6.10.0)
Requirement already satisfied: nbformat>=4.2.0 in /opt/conda/lib/python3.12/site-packages (from pycaret) (5.10.4)
Requirement already satisfied: cloudpickle in /opt/conda/lib/python3.12/site-packages (from pycaret) (3.1.2)
Collecting deprecation>=2.1.0 (from pycaret)
  Using cached deprecation-2.1.0-py2.py3-none-any.whl.metadata (4.6 kB)
Requirement already satisfied: xxhash in /opt/conda/lib/python3.12/site-packages (from pycaret) (3.7.0)
Collecting matplotlib<3.8.0 (from pycaret)
  Using cached matplotlib-3.7.5-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (5.7 kB)
Collecting scikit-plot>=0.3.7 (from pycaret)
  Using cached scikit_plot-0.3.7-py3-none-any.whl.metadata (7.1 kB)
Collecting yellowbrick>=1.4 (from pycaret)
  Using cached yellowbrick-1.5-py3-none-any.whl.metadata (7.7 kB)
Requirement already satisfied: plotly>=5.14.0 in /opt/conda/lib/python3.12/site-packages (from pycaret) (6.0.1)
Collecting kaleido>=0.2.1 (from pycaret)
  Using cached kaleido-1.3.0-py3-none-any.whl.metadata (5.6 kB)
Collecting schemdraw==0.15 (from pycaret)
  Using cached schemdraw-0.15-py3-none-any.whl.metadata (2.2 kB)
Collecting plotly-resampler>=0.8.3.1 (from pycaret)
  Using cached plotly_resampler-0.11.0-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: statsmodels>=0.12.1 in /opt/conda/lib/python3.12/site-packages (from pycaret) (0.14.6)
Collecting sktime==0.26.0 (from pycaret)
  Using cached sktime-0.26.0-py3-none-any.whl.metadata (29 kB)
Collecting tbats>=1.1.3 (from pycaret)
```

```
  Using cached tbats-1.1.3-py3-none-any.whl.metadata (3.8 kB)
Collecting pmdarima>=2.0.4 (from pycaret)
  Using cached pmdarima-2.1.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (8.5 kB)
Collecting wurlitzer (from pycaret)
  Using cached wurlitzer-3.1.1-py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: packaging in /opt/conda/lib/python3.12/site-packages (from sktime==0.26.0->pycaret) (25.0)
Collecting scikit-base<0.8.0 (from sktime==0.26.0->pycaret)
  Using cached scikit_base-0.7.8-py3-none-any.whl.metadata (8.8 kB)
Collecting scikit-learn>1.4.0 (from pycaret)
  Using cached scikit_learn-1.4.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (11 kB)
Requirement already satisfied: contourpy>=1.0.1 in /opt/conda/lib/python3.12/site-packages (from matplotlib<3.8.0->pycaret) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /opt/conda/lib/python3.12/site-packages (from matplotlib<3.8.0->pycaret) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /opt/conda/lib/python3.12/site-packages (from matplotlib<3.8.0->pycaret) (4.63.
0)
Requirement already satisfied: kiwisolver>=1.0.1 in /opt/conda/lib/python3.12/site-packages (from matplotlib<3.8.0->pycaret) (1.5.
0)
Requirement already satisfied: pillow>=6.2.0 in /opt/conda/lib/python3.12/site-packages (from matplotlib<3.8.0->pycaret) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/conda/lib/python3.12/site-packages (from matplotlib<3.8.0->pycaret) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /opt/conda/lib/python3.12/site-packages (from matplotlib<3.8.0->pycaret) (2.
9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.12/site-packages (from pandas<2.2.0->pycaret) (2024.2)
Requirement already satisfied: tzdata>=2022.1 in /opt/conda/lib/python3.12/site-packages (from pandas<2.2.0->pycaret) (2026.3)
Requirement already satisfied: threadpoolctl>=2.0.0 in /opt/conda/lib/python3.12/site-packages (from scikit-learn>1.4.0->pycaret)
(3.6.0)
Requirement already satisfied: patsy>=0.5.1 in /opt/conda/lib/python3.12/site-packages (from category-encoders>=2.4.0->pycaret) (1.
0.2)
INFO: pip is looking at multiple versions of category-encoders to determine which version is compatible with other requirements. Th
is could take a while.
Collecting category-encoders>=2.4.0 (from pycaret)
  Using cached category_encoders-2.9.0-py3-none-any.whl.metadata (7.9 kB)
  Using cached category_encoders-2.8.1-py3-none-any.whl.metadata (7.9 kB)
  Using cached category_encoders-2.8.0-py3-none-any.whl.metadata (7.9 kB)
  Using cached category_encoders-2.7.0-py3-none-any.whl.metadata (7.9 kB)
Collecting sklearn-compat<0.2,>=0.1.6 (from imbalanced-learn>=0.12.0->pycaret)
  Using cached sklearn_compat-0.1.6-py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: zipp>=0.5 in /opt/conda/lib/python3.12/site-packages (from importlib-metadata>=4.12.0->pycaret) (4.
1.0)
Requirement already satisfied: decorator in /opt/conda/lib/python3.12/site-packages (from ipython>=5.5.0->pycaret) (5.3.1)
Requirement already satisfied: jedi>=0.16 in /opt/conda/lib/python3.12/site-packages (from ipython>=5.5.0->pycaret) (0.19.2)
Requirement already satisfied: matplotlib-inline in /opt/conda/lib/python3.12/site-packages (from ipython>=5.5.0->pycaret) (0.2.2)
Requirement already satisfied: pexpect>4.3 in /opt/conda/lib/python3.12/site-packages (from ipython>=5.5.0->pycaret) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /opt/conda/lib/python3.12/site-packages (from ipython>=5.5.0->pycar
et) (3.0.52)
Requirement already satisfied: pygments>=2.4.0 in /opt/conda/lib/python3.12/site-packages (from ipython>=5.5.0->pycaret) (2.20.0)
Requirement already satisfied: stack_data in /opt/conda/lib/python3.12/site-packages (from ipython>=5.5.0->pycaret) (0.6.3)
Requirement already satisfied: traitlets>=5.13.0 in /opt/conda/lib/python3.12/site-packages (from ipython>=5.5.0->pycaret) (5.15.1)
Requirement already satisfied: wcwidth in /opt/conda/lib/python3.12/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=5.
5.0->pycaret) (0.8.2)
```

```
Requirement already satisfied: comm>=0.1.3 in /opt/conda/lib/python3.12/site-packages (from ipywidgets>=7.6.5->pycaret) (0.2.3)
Requirement already satisfied: widgetsnbextension~=4.0.14 in /opt/conda/lib/python3.12/site-packages (from ipywidgets>=7.6.5->pycar
et) (4.0.15)
Requirement already satisfied: jupyterlab_widgets~=3.0.15 in /opt/conda/lib/python3.12/site-packages (from ipywidgets>=7.6.5->pycar
et) (3.0.16)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in /opt/conda/lib/python3.12/site-packages (from jedi>=0.16->ipython>=5.5.0->pyc
aret) (0.8.7)
Collecting choreographer>=1.3.0 (from kaleido>=0.2.1->pycaret)
  Using cached choreographer-1.3.0-py3-none-any.whl.metadata (6.9 kB)
Collecting logistro>=1.0.8 (from kaleido>=0.2.1->pycaret)
  Using cached logistro-2.0.1-py3-none-any.whl.metadata (3.9 kB)
Requirement already satisfied: orjson>=3.10.15 in /opt/conda/lib/python3.12/site-packages (from kaleido>=0.2.1->pycaret) (3.11.9)
Requirement already satisfied: platformdirs>=4.3.6 in /opt/conda/lib/python3.12/site-packages (from choreographer>=1.3.0->kaleido>=
0.2.1->pycaret) (4.11.0)
Collecting simplejson>=3.19.3 (from choreographer>=1.3.0->kaleido>=0.2.1->pycaret)
  Using cached simplejson-4.1.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (3.8 kB)
Requirement already satisfied: fastjsonschema>=2.15 in /opt/conda/lib/python3.12/site-packages (from nbformat>=4.2.0->pycaret) (2.2
1.2)
Requirement already satisfied: jsonschema>=2.6 in /opt/conda/lib/python3.12/site-packages (from nbformat>=4.2.0->pycaret) (4.23.0)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in /opt/conda/lib/python3.12/site-packages (from nbformat>=4.2.0->pycare
t) (5.9.1)
Requirement already satisfied: attrs>=22.2.0 in /opt/conda/lib/python3.12/site-packages (from jsonschema>=2.6->nbformat>=4.2.0->pyc
aret) (26.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /opt/conda/lib/python3.12/site-packages (from jsonschema>=2.
6->nbformat>=4.2.0->pycaret) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in /opt/conda/lib/python3.12/site-packages (from jsonschema>=2.6->nbformat>=4.2.
0->pycaret) (0.37.0)
Requirement already satisfied: rpds-py>=0.7.1 in /opt/conda/lib/python3.12/site-packages (from jsonschema>=2.6->nbformat>=4.2.0->py
caret) (0.27.1)
Requirement already satisfied: llvmlite<0.49,>=0.48.0dev0 in /opt/conda/lib/python3.12/site-packages (from numba>=0.55.0->pycaret)
(0.48.0)
Requirement already satisfied: ptyprocess>=0.5 in /opt/conda/lib/python3.12/site-packages (from pexpect>4.3->ipython>=5.5.0->pycare
t) (0.7.0)
Requirement already satisfied: narwhals>=1.15.1 in /opt/conda/lib/python3.12/site-packages (from plotly>=5.14.0->pycaret) (2.24.0)
Requirement already satisfied: dash>=2.11.0 in /opt/conda/lib/python3.12/site-packages (from plotly-resampler>=0.8.3.1->pycaret)
(2.18.1)
Collecting tsdownsample>=0.1.3 (from plotly-resampler>=0.8.3.1->pycaret)
  Using cached tsdownsample-0.1.5.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (8.1 kB)
Collecting Flask<3.1,>=1.0.4 (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret)
  Using cached flask-3.0.3-py3-none-any.whl.metadata (3.2 kB)
Collecting Werkzeug<3.1 (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret)
  Using cached werkzeug-3.0.6-py3-none-any.whl.metadata (3.7 kB)
Collecting dash-html-components==2.0.0 (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret)
  Using cached dash_html_components-2.0.0-py3-none-any.whl.metadata (3.8 kB)
Collecting dash-core-components==2.0.0 (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret)
  Using cached dash_core_components-2.0.0-py3-none-any.whl.metadata (2.9 kB)
Collecting dash-table==5.0.0 (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret)
  Using cached dash_table-5.0.0-py3-none-any.whl.metadata (2.4 kB)
```

```
Requirement already satisfied: typing-extensions>=4.1.1 in /opt/conda/lib/python3.12/site-packages (from dash>=2.11.0->plotly-resam
pler>=0.8.3.1->pycaret) (4.16.0)
Requirement already satisfied: retrying in /opt/conda/lib/python3.12/site-packages (from dash>=2.11.0->plotly-resampler>=0.8.3.1->p
ycaret) (1.4.2)
Requirement already satisfied: nest-asyncio in /opt/conda/lib/python3.12/site-packages (from dash>=2.11.0->plotly-resampler>=0.8.3.
1->pycaret) (1.6.0)
Requirement already satisfied: setuptools in /opt/conda/lib/python3.12/site-packages (from dash>=2.11.0->plotly-resampler>=0.8.3.1-
>pycaret) (83.0.0)
Requirement already satisfied: itsdangerous>=2.1.2 in /opt/conda/lib/python3.12/site-packages (from Flask<3.1,>=1.0.4->dash>=2.11.0
->plotly-resampler>=0.8.3.1->pycaret) (2.2.0)
Requirement already satisfied: click>=8.1.3 in /opt/conda/lib/python3.12/site-packages (from Flask<3.1,>=1.0.4->dash>=2.11.0->plotl
y-resampler>=0.8.3.1->pycaret) (8.2.1)
Requirement already satisfied: blinker>=1.6.2 in /opt/conda/lib/python3.12/site-packages (from Flask<3.1,>=1.0.4->dash>=2.11.0->plo
tly-resampler>=0.8.3.1->pycaret) (1.9.0)
Collecting Cython!=0.29.18,!=0.29.31,>=0.29 (from pmdarima>=2.0.4->pycaret)
  Using cached cython-3.2.9-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (4.6 kB)
INFO: pip is looking at multiple versions of pmdarima to determine which version is compatible with other requirements. This could
take a while.
Collecting pmdarima>=2.0.4 (from pycaret)
  Using cached pmdarima-2.1.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (8.5 kB)
  Using cached pmdarima-2.0.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl.metadata (7.8 kB)
Requirement already satisfied: urllib3 in /opt/conda/lib/python3.12/site-packages (from pmdarima>=2.0.4->pycaret) (1.26.20)
INFO: pip is looking at multiple versions of pyod to determine which version is compatible with other requirements. This could take
a while.
Collecting pyod>=1.1.3 (from pycaret)
  Using cached pyod-3.6.3-py3-none-any.whl.metadata (58 kB)
  Using cached pyod-3.6.2-py3-none-any.whl.metadata (58 kB)
  Using cached pyod-3.6.1-py3-none-any.whl.metadata (59 kB)
  Using cached pyod-3.6.0-py3-none-any.whl.metadata (58 kB)
  Using cached pyod-3.5.4-py3-none-any.whl.metadata (57 kB)
  Using cached pyod-3.5.2-py3-none-any.whl.metadata (58 kB)
  Using cached pyod-3.5.1-py3-none-any.whl.metadata (58 kB)
INFO: pip is still looking at multiple versions of pyod to determine which version is compatible with other requirements. This coul
d take a while.
  Using cached pyod-3.5.0-py3-none-any.whl.metadata (57 kB)
  Using cached pyod-3.4.0-py3-none-any.whl.metadata (57 kB)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib<3.8.0->py
caret) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /opt/conda/lib/python3.12/site-packages (from requests>=2.27.1->pycaret)
(3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/site-packages (from requests>=2.27.1->pycaret) (3.18)
Requirement already satisfied: certifi>=2023.5.7 in /opt/conda/lib/python3.12/site-packages (from requests>=2.27.1->pycaret) (2026.
6.17)
Requirement already satisfied: executing>=1.2.0 in /opt/conda/lib/python3.12/site-packages (from stack_data->ipython>=5.5.0->pycare
t) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in /opt/conda/lib/python3.12/site-packages (from stack_data->ipython>=5.5.0->pycare
t) (3.0.2)
Requirement already satisfied: pure_eval in /opt/conda/lib/python3.12/site-packages (from stack_data->ipython>=5.5.0->pycaret) (0.
```

```
2.3)
Using cached pycaret-3.3.2-py3-none-any.whl (486 kB)
Using cached schemdraw-0.15-py3-none-any.whl (106 kB)
Using cached sktime-0.26.0-py3-none-any.whl (21.8 MB)
Using cached joblib-1.3.2-py3-none-any.whl (302 kB)
Using cached matplotlib-3.7.5-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (11.6 MB)
Using cached pandas-2.1.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (11.7 MB)
Using cached scikit_base-0.7.8-py3-none-any.whl (130 kB)
Using cached scikit_learn-1.4.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.2 MB)
Using cached scipy-1.11.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (35.8 MB)
Using cached category_encoders-2.7.0-py3-none-any.whl (85 kB)
Using cached deprecation-2.1.0-py2.py3-none-any.whl (11 kB)
Using cached imbalanced_learn-0.14.2-py3-none-any.whl (236 kB)
Using cached sklearn_compat-0.1.6-py3-none-any.whl (22 kB)
Using cached kaleido-1.3.0-py3-none-any.whl (55 kB)
Using cached choreographer-1.3.0-py3-none-any.whl (52 kB)
Using cached logistro-2.0.1-py3-none-any.whl (8.6 kB)
Using cached plotly_resampler-0.11.0-py3-none-any.whl (82 kB)
Using cached dash_core_components-2.0.0-py3-none-any.whl (3.8 kB)
Using cached dash_html_components-2.0.0-py3-none-any.whl (4.1 kB)
Using cached dash_table-5.0.0-py3-none-any.whl (3.9 kB)
Using cached flask-3.0.3-py3-none-any.whl (101 kB)
Using cached werkzeug-3.0.6-py3-none-any.whl (227 kB)
Using cached pmdarima-2.0.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl (2.3 MB)
Using cached cython-3.2.9-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (3.4 MB)
Using cached pyod-3.4.0-py3-none-any.whl (383 kB)
Using cached scikit_plot-0.3.7-py3-none-any.whl (33 kB)
Using cached simplejson-4.1.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (190 kB)
Using cached tbats-1.1.3-py3-none-any.whl (44 kB)
Using cached tsdownsample-0.1.5.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.3 MB)
Using cached yellowbrick-1.5-py3-none-any.whl (282 kB)
Using cached wurlitzer-3.1.1-py3-none-any.whl (8.6 kB)
Installing collected packages: dash-table, dash-html-components, dash-core-components, wurlitzer, Werkzeug, tsdownsample, simplejso
n, scipy, scikit-base, schemdraw, logistro, joblib, deprecation, Cython, scikit-learn, pandas, matplotlib, Flask, choreographer, ye
llowbrick, sktime, sklearn-compat, scikit-plot, pyod, kaleido, pmdarima, plotly-resampler, imbalanced-learn, category-encoders, tba
ts, pycaret
  Attempting uninstall: Werkzeug
    Found existing installation: Werkzeug 3.1.8
    Uninstalling Werkzeug-3.1.8:
      Successfully uninstalled Werkzeug-3.1.8
  Attempting uninstall: scipy━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━  6/31 [simplejson]
    Found existing installation: scipy 1.16.3━━━━━━━━━━━━━━━━━  6/31 [simplejson]
    Uninstalling scipy-1.16.3:━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━  6/31 [simplejson]
      Successfully uninstalled scipy-1.16.3━━━━━━━━━━━━━━━━━━  6/31 [simplejson]
  Attempting uninstall: joblib[0m━━━━━━━━━━━━━━━━━━━━━━━━━━━  9/31 [schemdraw]e]
    Found existing installation: joblib 1.5.3━━━━━━━━━━━━━━━━  9/31 [schemdraw]
    Uninstalling joblib-1.5.3:[90m━━━━━━━━━━━━━━━━━━━━━━━━━━━  9/31 [schemdraw]
      Successfully uninstalled joblib-1.5.3━━━━━━━━━━━━━━━━━━  9/31 [schemdraw]
```

```
  Attempting uninstall: scikit-learn0m━━━━━━━━━━━━━━━━━━━━━━━━ 13/31 [Cython]
    Found existing installation: scikit-learn 1.7.2━━━━━━━━━━━ 13/31 [Cython]
    Uninstalling scikit-learn-1.7.2:90m━━━━━━━━━━━━━━━━━━━━━━ 13/31 [Cython]
      Successfully uninstalled scikit-learn-1.7.2━━━━━━━━━━━━ 13/31 [Cython]
  Attempting uninstall: pandas ━━━━━━━━━━━━━━━━━━━━━━ 14/31 [scikit-learn]
    Found existing installation: pandas 2.3.3━━━━━━━━━━━━━━━ 14/31 [scikit-learn]
    Uninstalling pandas-2.3.3:m ━━━━━━━━━━━━━━━━━━━━ 15/31 [pandas]n]
      Successfully uninstalled pandas-2.3.3━━━━━━━━━━━━━━━━━ 15/31 [pandas]
  Attempting uninstall: matplotlib0m ━━━━━━━━━━━━━━━━━━ 15/31 [pandas]
    Found existing installation: matplotlib 3.10.9━━━━━━━━━━ 15/31 [pandas]
    Uninstalling matplotlib-3.10.9:0m━━━━━━━━━━━━━━━━━━ 15/31 [pandas]
      Successfully uninstalled matplotlib-3.10.9━━━━━━━━━━━━━━ 16/31 [matplotlib]
  Attempting uninstall: Flask[0m- ━━━━━━━━━━━━━━━━━ 16/31 [matplotlib]
    Found existing installation: Flask 3.1.3━━━━━━━━━━━━━━━ 16/31 [matplotlib]
    Uninstalling Flask-3.1.3:[91m- ━━━━━━━━━━━━━━━━━ 16/31 [matplotlib]
      Successfully uninstalled Flask-3.1.3m━━━━━━━━━━━━━━━━━ 16/31 [matplotlib]
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 31/31 [pycaret]0/31 [pycaret]-encoders]
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the so
urce of the following dependency conflicts.
autogluon-multimodal 1.5.0 requires nvidia-ml-py3<8.0,>=7.352.0, which is not installed.
autogluon-timeseries 1.5.0 requires chronos-forecasting<2.4,>=2.2.2, which is not installed.
autogluon-timeseries 1.5.0 requires einops<1,>=0.7, which is not installed.
autogluon-timeseries 1.5.0 requires peft<0.18,>=0.13.0, which is not installed.
skops 0.14.0 requires prettytable>=3.9, which is not installed.
autogluon-common 1.5.0 requires pyarrow<21.0.0,>=7.0.0, but you have pyarrow 21.0.0 which is incompatible.
autogluon-multimodal 1.5.0 requires fsspec[http]<=2025.3, but you have fsspec 2026.6.0 which is incompatible.
sagemaker-studio 1.1.26 requires pandas<3.0.0,>=2.3.2, but you have pandas 2.1.4 which is incompatible.
sparkmagic 0.21.0 requires pandas<2.0.0,>=0.17.1, but you have pandas 2.1.4 which is incompatible.
Successfully installed Cython-3.2.9 Flask-3.0.3 Werkzeug-3.0.6 category-encoders-2.7.0 choreographer-1.3.0 dash-core-components-2.
0.0 dash-html-components-2.0.0 dash-table-5.0.0 deprecation-2.1.0 imbalanced-learn-0.14.2 joblib-1.3.2 kaleido-1.3.0 logistro-2.0.1
matplotlib-3.7.5 pandas-2.1.4 plotly-resampler-0.11.0 pmdarima-2.0.4 pycaret-3.3.2 pyod-3.4.0 schemdraw-0.15 scikit-base-0.7.8 scik
it-learn-1.4.2 scikit-plot-0.3.7 scipy-1.11.4 simplejson-4.1.1 sklearn-compat-0.1.6 sktime-0.26.0 tbats-1.1.3 tsdownsample-0.1.5.1
wurlitzer-3.1.1 yellowbrick-1.5
```

In [8]:

```python
from pycaret.regression import setup, compare_models, pull

# 1. Configurar entorno
experiment = setup(data=df, target='ivc_n', session_id=123)

# 2. Entrenar y comparar
best_model = compare_models(sort='R2')

# 3. Capturar la tabla de métricas generada automáticamente
resultados_metricas = pull()

# Mostrar la tabla para confirmar
display(resultados_metricas)
```

```
# (Opcional) Exportar la tabla para el documento de tu tesis
resultados_metricas.to_csv('metricas_modelos_tesis.csv', index=True)
```

	Description	Value
0	Session id	123
1	Target	ivc_n
2	Target type	Regression
3	Original data shape	(43408, 59)
4	Transformed data shape	(43408, 77)
5	Transformed train set shape	(30385, 77)
6	Transformed test set shape	(13023, 77)
7	Numeric features	56
8	Categorical features	2
9	Rows with missing values	0.2%
10	Preprocess	True
11	Imputation type	simple
12	Numeric imputation	mean
13	Categorical imputation	mode
14	Maximum one-hot encoding	25
15	Encoding method	None
16	Fold Generator	KFold
17	Fold Number	10
18	CPU Jobs	-1
19	Use GPU	False
20	Log Experiment	False
21	Experiment Name	reg-default-name
22	USI	38a2

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE	TT (Sec)
lightgbm	Light Gradient Boosting Machine	0.0159	0.0007	0.0260	0.8856	0.0224	6.1264	45.6420
et	Extra Trees Regressor	0.0162	0.0007	0.0267	0.8795	0.0228	7.3890	9.3400
rf	Random Forest Regressor	0.0161	0.0007	0.0267	0.8794	0.0229	7.3240	16.4210
gbr	Gradient Boosting Regressor	0.0187	0.0008	0.0291	0.8566	0.0249	7.3809	4.7100
dt	Decision Tree Regressor	0.0234	0.0015	0.0390	0.7423	0.0334	9.5914	0.4190
ada	AdaBoost Regressor	0.0376	0.0022	0.0467	0.6305	0.0409	9.2521	2.7320
knn	K Neighbors Regressor	0.0315	0.0023	0.0477	0.6156	0.0405	9.8032	0.2590
br	Bayesian Ridge	0.0350	0.0024	0.0489	0.5952	0.0413	5.6486	0.1790
ridge	Ridge Regression	0.0351	0.0024	0.0490	0.5936	0.0414	6.8489	0.0960
lr	Linear Regression	0.0349	0.0024	0.0492	0.5914	0.0415	3.5765	0.4470
omp	Orthogonal Matching Pursuit	0.0519	0.0045	0.0673	0.2345	0.0562	7.9857	0.0920
en	Elastic Net	0.0592	0.0051	0.0714	0.1370	0.0613	8.7921	0.1540
lasso	Lasso Regression	0.0646	0.0058	0.0765	0.0118	0.0658	9.0180	0.1040
llar	Lasso Least Angle	0.0646	0.0058	0.0765	0.0118	0.0658	9.0180	0.0980

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE	TT (Sec)
	Regression							
dummy	Dummy Regressor	0.0655	0.0059	0.0769	-0.0004	0.0665	13.6596	0.0880
huber	Huber Regressor	0.0595	0.0086	0.0898	-0.4425	0.0662	8.2912	0.5340
par	Passive Aggressive Regressor	0.0798	0.5281	0.4241	-89.0808	0.1045	54.9762	0.1170
lar	Least Angle Regression	17685575.6643	63676465958907336.0000	98080468.1555	-10768371074793566208.0000	11.7278	456984169.5682	0.1020

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE	TT (Sec)
lightgbm	Light Gradient Boosting Machine	1.590000e-02	7.000000e-04	2.600000e-02	8.856000e-01	0.0224	6.126400e+00	45.642
et	Extra Trees Regressor	1.620000e-02	7.000000e-04	2.670000e-02	8.795000e-01	0.0228	7.389000e+00	9.340
rf	Random Forest Regressor	1.610000e-02	7.000000e-04	2.670000e-02	8.794000e-01	0.0229	7.324000e+00	16.421
gbr	Gradient Boosting Regressor	1.870000e-02	8.000000e-04	2.910000e-02	8.566000e-01	0.0249	7.380900e+00	4.710
dt	Decision Tree Regressor	2.340000e-02	1.500000e-03	3.900000e-02	7.423000e-01	0.0334	9.591400e+00	0.419
ada	AdaBoost Regressor	3.760000e-02	2.200000e-03	4.670000e-02	6.305000e-01	0.0409	9.252100e+00	2.732
knn	K Neighbors Regressor	3.150000e-02	2.300000e-03	4.770000e-02	6.156000e-01	0.0405	9.803200e+00	0.259
br	Bayesian Ridge	3.500000e-02	2.400000e-03	4.890000e-02	5.952000e-01	0.0413	5.648600e+00	0.179
ridge	Ridge Regression	3.510000e-02	2.400000e-03	4.900000e-02	5.936000e-01	0.0414	6.848900e+00	0.096
lr	Linear Regression	3.490000e-02	2.400000e-03	4.920000e-02	5.914000e-01	0.0415	3.576500e+00	0.447
omp	Orthogonal Matching Pursuit	5.190000e-02	4.500000e-03	6.730000e-02	2.345000e-01	0.0562	7.985700e+00	0.092
en	Elastic Net	5.920000e-02	5.100000e-03	7.140000e-02	1.370000e-01	0.0613	8.792100e+00	0.154
lasso	Lasso Regression	6.460000e-02	5.800000e-03	7.650000e-02	1.180000e-02	0.0658	9.018000e+00	0.104
llar	Lasso Least Angle Regression	6.460000e-02	5.800000e-03	7.650000e-02	1.180000e-02	0.0658	9.018000e+00	0.098
dummy	Dummy Regressor	6.550000e-02	5.900000e-03	7.690000e-02	-4.000000e-04	0.0665	1.365960e+01	0.088
huber	Huber Regressor	5.950000e-02	8.600000e-03	8.980000e-02	-4.425000e-01	0.0662	8.291200e+00	0.534
par	Passive Aggressive Regressor	7.980000e-02	5.281000e-01	4.241000e-01	-8.908080e+01	0.1045	5.497620e+01	0.117
lar	Least Angle Regression	1.768558e+07	6.367647e+16	9.808047e+07	-1.076837e+19	11.7278	4.569842e+08	0.102

MODELO DE MACHINE LEARNING

Vamos a complementar tu pipeline con scikit-learn puro, usando el mejor modelo que salga de pycaret como base.

In [9]:
```
from pycaret.regression import setup, create_model, pull, get_config

experiment = setup(data=df, target='ivc_n', session_id=123)

# verbose=False → silencia la salida de PyCaret
# verbosity=-1  → silencia los logs internos de LightGBM (Auto-choosing col-wise...)
best_model = create_model('lightgbm', verbose=False, verbosity=-1)
```

```
metricas_lightgbm = pull()
display(metricas_lightgbm)
metricas_lightgbm.to_csv('metricas_lightgbm_tesis.csv', index=True)

print(type(best_model))
print(best_model)
```

	Description	Value
0	Session id	123
1	Target	ivc_n
2	Target type	Regression
3	Original data shape	(43408, 59)
4	Transformed data shape	(43408, 77)
5	Transformed train set shape	(30385, 77)
6	Transformed test set shape	(13023, 77)
7	Numeric features	56
8	Categorical features	2
9	Rows with missing values	0.2%
10	Preprocess	True
11	Imputation type	simple
12	Numeric imputation	mean
13	Categorical imputation	mode
14	Maximum one-hot encoding	25
15	Encoding method	None
16	Fold Generator	KFold
17	Fold Number	10
18	CPU Jobs	-1
19	Use GPU	False
20	Log Experiment	False
21	Experiment Name	reg-default-name
22	USI	fe25

Fold	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	0.0159	0.0007	0.0258	0.8879	0.0223	0.1904
1	0.0159	0.0007	0.0266	0.8812	0.0229	0.1927
2	0.0162	0.0007	0.0268	0.8766	0.0232	0.1993
3	0.0158	0.0006	0.0254	0.8899	0.0222	0.1921
4	0.0155	0.0006	0.0252	0.8927	0.0218	0.1935
5	0.0160	0.0006	0.0248	0.8919	0.0216	0.1856
6	0.0164	0.0008	0.0285	0.8698	0.0237	0.3831
7	0.0155	0.0006	0.0251	0.8974	0.0216	0.1926
8	0.0159	0.0007	0.0261	0.8830	0.0224	0.1905
9	0.0159	0.0007	0.0257	0.8860	0.0223	59.3438
Mean	0.0159	0.0007	0.0260	0.8856	0.0224	6.1264
Std	0.0003	0.0001	0.0010	0.0078	0.0007	17.7392

```
<class 'lightgbm.sklearn.LGBMRegressor'>
LGBMRegressor(n_jobs=-1, random_state=123, verbosity=-1)
```

In [15]:
```
metricas_tuned = pull()
display(metricas_tuned)
```

Fold	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	0.0172	0.0008	0.0276	0.8673	0.024	0.2081
1	0.0176	0.0008	0.0283	0.8706	0.024	0.2084
Mean	0.0174	0.0008	0.0280	0.8690	0.024	0.2083
Std	0.0002	0.0000	0.0004	0.0017	0.000	0.0002

In [13]:
```
# Opción de emergencia: tuning sobre una muestra, luego reentrenar completo
df_muestra = df.sample(frac=0.25, random_state=123)  # 25% de los datos SOLO para encontrar hiperparámetros

experiment_tuning = setup(data=df_muestra, target='ivc_n', session_id=123)
```

```
best_model_muestra = create_model('lightgbm', verbose=False, verbosity=-1)

tuned_model = tune_model(
    best_model_muestra,
    optimize='RMSE', n_iter=8, fold=2,
    search_library='scikit-learn', search_algorithm='random',
    choose_better=True,
    custom_grid={
        'n_estimators': [100, 200], 'num_leaves': [20, 31],
        'learning_rate': [0.1, 0.2], 'max_depth': [8, 15], 'n_jobs': [-1],
    },
    verbose=False
)
print(tuned_model.get_params())  # anota los hiperparámetros ganadores
```

	Description	Value
0	Session id	123
1	Target	ivc_n
2	Target type	Regression
3	Original data shape	(10852, 59)
4	Transformed data shape	(10852, 77)
5	Transformed train set shape	(7596, 77)
6	Transformed test set shape	(3256, 77)
7	Numeric features	56
8	Categorical features	2
9	Rows with missing values	0.2%
10	Preprocess	True
11	Imputation type	simple
12	Numeric imputation	mean
13	Categorical imputation	mode
14	Maximum one-hot encoding	25
15	Encoding method	None
16	Fold Generator	KFold
17	Fold Number	10
18	CPU Jobs	-1
19	Use GPU	False
20	Log Experiment	False
21	Experiment Name	reg-default-name
22	USI	1e0e


```
{'boosting_type': 'gbdt', 'class_weight': None, 'colsample_bytree': 1.0, 'importance_type': 'split', 'learning_rate': 0.1, 'max_dep
th': 15, 'min_child_samples': 20, 'min_child_weight': 0.001, 'min_split_gain': 0.0, 'n_estimators': 200, 'n_jobs': -1, 'num_leave
s': 20, 'objective': None, 'random_state': 123, 'reg_alpha': 0.0, 'reg_lambda': 0.0, 'subsample': 1.0, 'subsample_for_bin': 200000,
'subsample_freq': 0, 'verbosity': -1}
```

In [16]:

```python
from pycaret.regression import setup, create_model, pull, finalize_model, save_model, get_config

# 1. Setup con el dataset COMPLETO
experiment = setup(data=df, target='ivc_n', session_id=123)

# 2. Crear el modelo con los hiperparámetros exactos hallados en el tuning
modelo_final = create_model(
    'lightgbm',
    boosting_type='gbdt',
    max_depth=15,
    n_estimators=200,
    num_leaves=20,
    learning_rate=0.1,
    n_jobs=-1,
    random_state=123,
    verbosity=-1,
    verbose=False
)

metricas_finales = pull()
display(metricas_finales)
metricas_finales.to_csv('metricas_hiperparametros_finales_tesis.csv', index=True)

print(modelo_final)

# 3. Comparación directa
print("\n--- Comparación ---")
print("Modelo base (default):      MAE=0.0159  RMSE=0.0260  R2=0.8856")
print(f"Modelo con hiperparámetros:  MAE={metricas_finales.loc['Mean','MAE']}  "
      f"RMSE={metricas_finales.loc['Mean','RMSE']}  R2={metricas_finales.loc['Mean','R2']}")
```

	Description	Value
0	Session id	123
1	Target	ivc_n
2	Target type	Regression
3	Original data shape	(43408, 59)
4	Transformed data shape	(43408, 77)
5	Transformed train set shape	(30385, 77)
6	Transformed test set shape	(13023, 77)
7	Numeric features	56
8	Categorical features	2
9	Rows with missing values	0.2%
10	Preprocess	True
11	Imputation type	simple
12	Numeric imputation	mean
13	Categorical imputation	mode
14	Maximum one-hot encoding	25
15	Encoding method	None
16	Fold Generator	KFold
17	Fold Number	10
18	CPU Jobs	-1
19	Use GPU	False
20	Log Experiment	False
21	Experiment Name	reg-default-name
22	USI	e6f2

Fold	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	0.0157	0.0007	0.0255	0.8902	0.0221	0.1900
1	0.0157	0.0007	0.0264	0.8832	0.0227	0.1903
2	0.0159	0.0007	0.0263	0.8811	0.0228	0.1938
3	0.0153	0.0006	0.0250	0.8938	0.0218	0.1884
4	0.0150	0.0006	0.0246	0.8976	0.0214	0.1904
5	0.0155	0.0006	0.0243	0.8964	0.0212	0.1800
6	0.0159	0.0008	0.0275	0.8786	0.0231	0.3599
7	0.0153	0.0006	0.0249	0.8995	0.0214	0.1924
8	0.0155	0.0007	0.0257	0.8869	0.0221	0.1870
9	0.0154	0.0006	0.0252	0.8903	0.0219	65.5828
Mean	0.0155	0.0007	0.0255	0.8898	0.0220	6.7455
Std	0.0003	0.0000	0.0009	0.0068	0.0006	19.6125

```
LGBMRegressor(max_depth=15, n_estimators=200, n_jobs=-1, num_leaves=20,
              random_state=123, verbosity=-1)

--- Comparación ---
Modelo base (default):      MAE=0.0159  RMSE=0.0260  R2=0.8856
Modelo con hiperparámetros:  MAE=0.0155  RMSE=0.0255  R2=0.8898
```

Comparar modelo acorde train y test

In [19]:
```python
from pycaret.regression import get_config
from sklearn.model_selection import train_test_split
from sklearn.base import clone
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np
import pandas as pd

X = get_config('X')
y = get_config('y')
print("Shape X, y:", X.shape, y.shape)  # debería confirmar (43408, 76) y (43408,)

splits = {'70/30': 0.30, '80/20': 0.20, '90/10': 0.10, '60/40': 0.40}
resultados_splits = []
```

```
modelos_por_split = {}

for nombre_split, test_size in splits.items():
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=123
    )
    modelo = clone(modelo_final)
    modelo.fit(X_train, y_train)
    y_pred = modelo.predict(X_test)

    resultados_splits.append({
        'split': nombre_split, 'n_train': len(X_train), 'n_test': len(X_test),
        'R2': r2_score(y_test, y_pred),
        'MAE': mean_absolute_error(y_test, y_pred),
        'MSE': mean_squared_error(y_test, y_pred),
        'RMSE': np.sqrt(mean_squared_error(y_test, y_pred))
    })
    modelos_por_split[nombre_split] = modelo

df_splits = pd.DataFrame(resultados_splits)
display(df_splits)
df_splits.to_csv('comparacion_splits_hiperparametros_finales_tesis.csv', index=False)

mejor_split = df_splits.loc[df_splits['RMSE'].idxmin(), 'split']
print(f"Mejor partición según RMSE: {mejor_split}")
```

```
Shape X, y: (43408, 58) (43408,)
```

	split	n_train	n_test	R2	MAE	MSE	RMSE
0	70/30	30385	13023	0.892391	0.015680	0.000644	0.025370
1	80/20	34726	8682	0.892157	0.015652	0.000647	0.025446
2	90/10	39067	4341	0.899661	0.015419	0.000609	0.024681
3	60/40	26044	17364	0.890834	0.015690	0.000651	0.025521

```
Mejor partición según RMSE: 90/10
```

El proceso de ajuste de hiperparámetros (fine-tuning) aplicado al modelo LightGBM produjo una mejora consistente y estadísticamente relevante respecto al modelo base. Al evaluar el modelo optimizado (max_depth=15, num_leaves=20, n_estimators=200) bajo cuatro esquemas de partición train/test (60/40, 70/30, 80/20 y 90/10), se observó un desempeño estable con coeficientes de determinación (R^2) en el rango de 0.891 a 0.900, frente al 0.8856 obtenido por el modelo con hiperparámetros por defecto — una mejora de entre 0.5 y 1.4 puntos porcentuales, acompañada de una reducción sistemática del error (RMSE entre 0.0247 y 0.0255, frente a 0.0260 del modelo base).

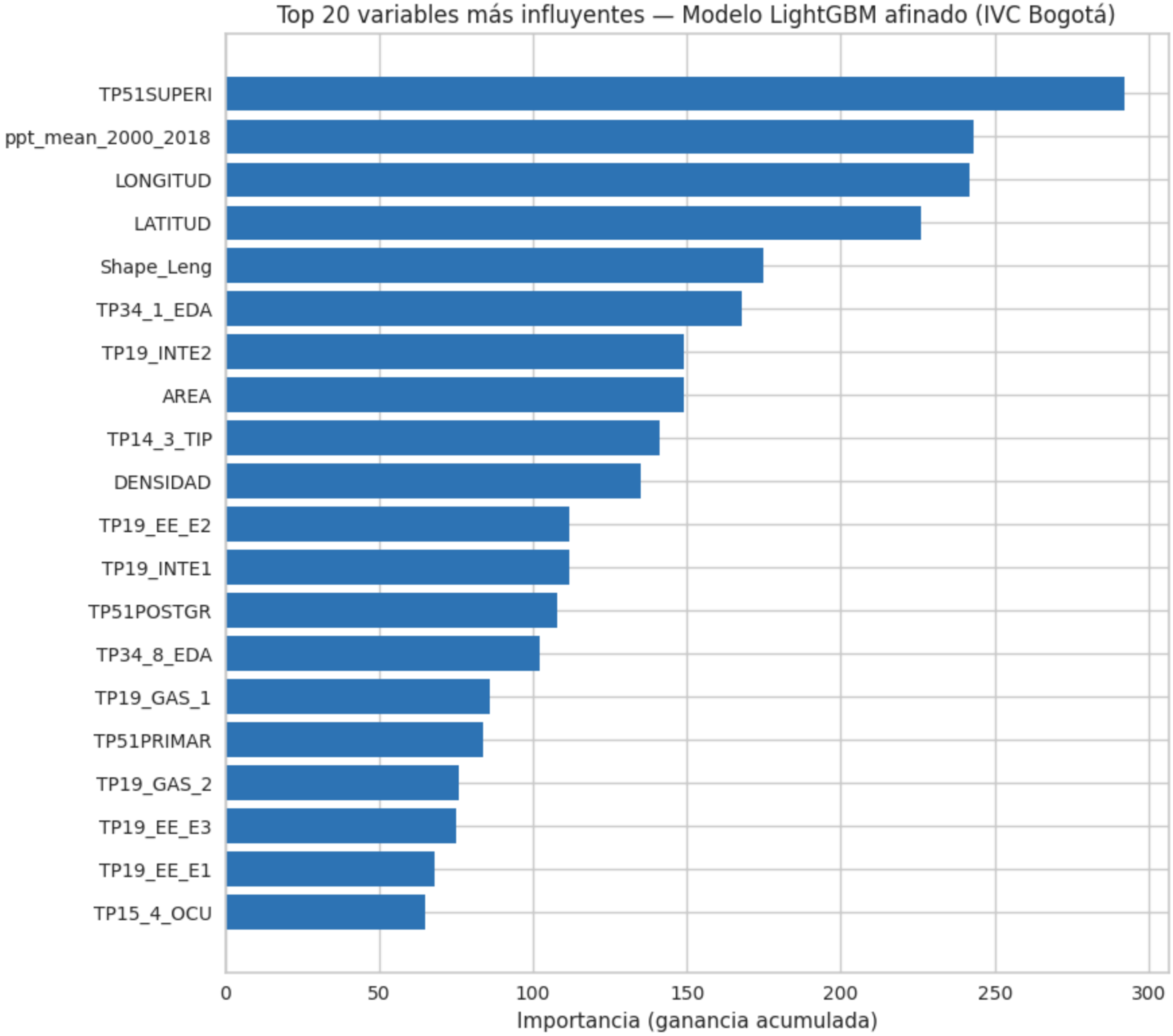
La baja dispersión de resultados entre las distintas proporciones de partición (variación de apenas 0.9 puntos de R^2 entre el esquema más exigente —60/40— y el más favorable —90/10—) constituye evidencia de que el modelo no depende de una configuración particular de datos para

generalizar, sino que captura relaciones estructurales robustas entre las variables predictoras (socioeconómicas, catastrales y climáticas) y el Índice de Vulnerabilidad Climática. Este comportamiento es consistente con los resultados obtenidos en la validación cruzada k-fold reportada previamente, reforzando la fiabilidad del modelo antes de proceder a su finalización y despliegue.

En términos prácticos, estos resultados validan la decisión metodológica de restringir la complejidad del modelo (menor num_leaves respecto al valor por defecto), evidenciando que una configuración más conservadora no solo mejora la precisión predictiva, sino que previsiblemente favorece la capacidad de generalización del modelo — un atributo especialmente relevante de cara al ejercicio de transferencia espacial hacia Medellín y Cali, donde el modelo deberá operar sobre datos con distribuciones distintas a las observadas durante el entrenamiento en Bogotá.

In [21]:
```python
import pandas as pd
import matplotlib.pyplot as plt

# Nombres de features tal como los vio el modelo (ya con one-hot encoding aplicado por pycaret)
nombres_features = modelo_final.feature_name_

importancias = pd.DataFrame({
    'feature': nombres_features,
    'importancia': modelo_final.feature_importances_
}).sort_values('importancia', ascending=False).reset_index(drop=True)

display(importancias.head(20))
importancias.to_csv('feature_importance_modelo_final_tesis.csv', index=False)

# Gráfico de barras horizontal — Top 20
top_20 = importancias.head(20).sort_values('importancia', ascending=True)

plt.figure(figsize=(9, 8))
plt.barh(top_20['feature'], top_20['importancia'], color='#2E75B6')
plt.xlabel('Importancia (ganancia acumulada)')
plt.title('Top 20 variables más influyentes — Modelo LightGBM afinado (IVC Bogotá)')
plt.tight_layout()
plt.savefig('top20_feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()
```

	feature	importancia
0	TP51SUPERI	292
1	ppt_mean_2000_2018	243
2	LONGITUD	242
3	LATITUD	226
4	Shape_Leng	175
5	TP34_1_EDA	168
6	TP19_INTE2	149
7	AREA	149
8	TP14_3_TIP	141
9	DENSIDAD	135
10	TP19_EE_E2	112
11	TP19_INTE1	112
12	TP51POSTGR	108
13	TP34_8_EDA	102
14	TP19_GAS_1	86
15	TP51PRIMAR	84
16	TP19_GAS_2	76
17	TP19_EE_E3	75
18	TP19_EE_E1	68
19	TP15_4_OCU	65


Top 20 variables más influyentes — Modelo LightGBM afinado (IVC Bogotá)
TP51SUPERI
ppt_mean_2000_2018
LONGITUD
LATITUD
Shape_Leng
TP34_1_EDA
TP19_INTE2
AREA
TP14_3_TIP
DENSIDAD
TP19_EE_E2
TP19_INTE1
TP51POSTGR
TP34_8_EDA
TP19_GAS_1
TP51PRIMAR
TP19_GAS_2
TP19_EE_E3
TP19_EE_E1
TP15_4_OCU
0
50
100
150
200
250
300
Importancia (ganancia acumulada)

In [22]:
```
# Agrupar importancia por variable original (sumando categorías del one-hot encoding)
importancias['feature_original'] = importancias['feature'].str.split('_').str[:2].str.join('_')
# Ajusta el split según cómo pycaret nombró tus columnas categóricas — revisa importancias['feature'] primero

importancia_agrupada = importancias.groupby('feature_original')['importancia'].sum().sort_values(ascending=False).head(20)
display(importancia_agrupada)
```

```
feature_original
TP19_EE       350
TP51SUPERI    292
ppt_mean      243
LONGITUD      242
LATITUD       226
Shape_Leng    175
TP34_1        168
TP19_GAS      167
AREA          149
TP19_INTE2    149
TP14_3        141
DENSIDAD      135
TP19_ALC      117
TP19_INTE1    112
TP51POSTGR    108
TP34_8        102
TP51PRIMAR     84
TP19_ACU       78
TP15_4         65
TP51SECUND     63
Name: importancia, dtype: int32
```

In [23]:
```
from pycaret.regression import finalize_model, save_model

modelo_produccion = finalize_model(modelo_final)
save_model(modelo_produccion, 'modelo_ivc_bogota_final_tuned')
```

```
Transformation Pipeline and Model Successfully Saved
```

Out[23]:
```
(Pipeline(memory=Memory(location=None),
          steps=[('numerical_imputer',
                  TransformerWrapper(include=['CD_LC_CM', 'AREA', 'LATITUD',
                                              'LONGITUD', 'Shape_Leng',
                                              'Shape_Area', 'TP27_PERSO',
                                              'TVIVIENDA', 'TP16_HOG',
                                              'DENSIDAD', 'TP34_1_EDA',
                                              'TP34_8_EDA', 'TP34_9_EDA',
                                              'TP51_13_ED', 'TP51POSTGR',
                                              'TP51PRIMAR', 'TP51SECUND',
                                              'TP51SUPERI', 'TP51_99_ED',
                                              'TP14_6_TIP', 'TP15_4_OCU',
                                              '...
                                     transformer=SimpleImputer(strategy='most_frequent'))),
                 ('onehot_encoding',
                  TransformerWrapper(include=['NMB_LC_CM', 'TP_LC_CM'],
                                     transformer=OneHotEncoder(cols=['NMB_LC_CM',
                                                                     'TP_LC_CM'],
                                                               handle_missing='return_nan',
                                                               use_cat_names=True))),
                 ('actual_estimator',
                  LGBMRegressor(max_depth=15, n_estimators=200, n_jobs=-1,
                                num_leaves=20, random_state=123,
                                verbosity=-1))]),
 'modelo_ivc_bogota_final_tuned.pkl')
```

In [24]:
```
import joblib
import awswrangler as wr

joblib.dump(modelo_produccion, 'modelo_ivc_bogota_final_tuned.pkl')
wr.s3.upload(local_file='modelo_ivc_bogota_final_tuned.pkl',
             path='s3://databogota/modelos/modelo_ivc_bogota_final_tuned.pkl')
```

PPREDICCIONES BOGOTÁ

Paso 1: Cargar los datos y generar las predicciones

In [25]:
```
import awswrangler as wr
import pandas as pd
from pycaret.regression import predict_model

# Cargar el dataset (mismo usado en el entrenamiento)
df_bogota = wr.s3.read_csv('s3://bogotamanzanas/bogota_ivc_thesis_ready.csv')
print(df_bogota.shape)
print(df_bogota.columns.tolist())
```

```
# Generar predicciones con el modelo final
predicciones = predict_model(modelo_produccion, data=df_bogota)
predicciones = predicciones.rename(columns={'prediction_label': 'IVC_predicho'})

display(predicciones[['IVC_predicho']].describe())
```

```
(43408, 62)
['CD_LC_CM', 'NMB_LC_CM', 'TP_LC_CM', 'AREA', 'LATITUD', 'LONGITUD', 'Shape_Leng', 'Shape_Area', 'TP27_PERSO', 'TVIVIENDA', 'TP16_HOG', 'DENSIDAD', 'TP34_1_EDA', 'TP34_8_EDA', 'TP34_9_EDA', 'TP51_13_ED', 'TP51POSTGR', 'TP51PRIMAR', 'TP51SECUND', 'TP51SUPERI', 'TP51_99_ED', 'TP14_6_TIP', 'TP15_4_OCU', 'TP9_3_10_N', 'TP9_3_6_NO', 'TP14_3_TIP', 'TP15_1_OCU', 'TP15_2_OCU', 'TP15_3_OCU', 'TP9_3_8_NO', 'TP9_3_9_NO', 'TP9_2_4_MI', 'TP9_3_4_NO', 'TP19_EE_2', 'TP19_ACU_2', 'TP19_ALC_2', 'TP19_GAS_2', 'TP19_RECB2', 'TP19_INTE2', 'TP19_GAS_9', 'TP19_INTE9', 'TP19_EE_1', 'TP19_ACU_1', 'TP19_ALC_1', 'TP19_GAS_1', 'TP19_RECB1', 'TP19_INTE1', 'TP19_EE_E1', 'TP19_EE_E2', 'TP19_EE_E3', 'TP19_EE_E4', 'TP19_EE_E5', 'TP19_EE_E6', 'TP19_EE_E9', 'TP9_1_USO', 'TP9_2_USO', 'TP9_3_USO', 'ppt_mean_2000_2018', 'ivc_formula', 'ivc_n', 'categoria_ivc', 'perfil_vulnerabilidad']
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	Light Gradient Boosting Machine	0.0139	0.0005	0.0222	0.9176	0.0194	4.4160

	IVC_predicho
count	43408.000000
mean	0.148473
std	0.072920
min	-0.017797
25%	0.085229
50%	0.143736
75%	0.211305
max	0.841165

Paso 2: Descargar e inspeccionar el shapefile

In [30]:
```
import geopandas as gpd
import zipfile
import os

# Descargar el .zip desde S3
wr.s3.download('s3://bogotamanzanas/manzana.0225.shp.zip', local_file='manzana.0225.shp.zip')

# Descomprimir
with zipfile.ZipFile('manzana.0225.shp.zip', 'r') as z:
    z.extractall('manzana_shp')
```

```
# Buscar el archivo .shp dentro de la carpeta descomprimida
shp_files = [f for f in os.listdir('manzana_shp') if f.endswith('.shp')]
print(shp_files)

gdf_manzanas = gpd.read_file(f'manzana_shp/{shp_files[0]}')
print(gdf_manzanas.shape)
print(gdf_manzanas.columns.tolist())
print(gdf_manzanas.head())
```

```
['MANZANA.shp']
ERROR 1: PROJ: proj_create_from_database: Open of /home/sagemaker-user/.conda/envs/pycaret_env/share/proj failed
(44057, 5)
['MANCODIGO', 'SECCODIGO', 'SHAPE_Leng', 'SHAPE_Area', 'geometry']
   MANCODIGO SECCODIGO  SHAPE_Leng    SHAPE_Area  \
0  004501049    004501    0.004340  6.849815e-07
1  004537005    004537    0.002015  2.452513e-07
2  002569081    002569    0.000975  4.103821e-08
3  004537006    004537    0.002056  2.603283e-07
4  004537009    004537    0.002006  2.393863e-07

                                            geometry
0  POLYGON ((-74.13655 4.62794, -74.13656 4.62785...
1  POLYGON ((-74.17189 4.61561, -74.17174 4.6154,...
2  POLYGON ((-74.16526 4.56347, -74.16527 4.56344...
3  POLYGON ((-74.17212 4.61557, -74.17233 4.6154,...
4  POLYGON ((-74.17146 4.61533, -74.17147 4.61532...
```

In [31]:
```
import geopandas as gpd
print(gpd.__version__)
```

```
1.1.4
```

Cargar el CSV y generar las predicciones

In [ ]:
```
import awswrangler as wr
import pandas as pd
from pycaret.regression import predict_model

df_bogota = wr.s3.read_csv('s3://bogotamanzanas/bogota_ivc_thesis_ready.csv')
print(df_bogota.shape)
print(df_bogota.columns.tolist())

predicciones = predict_model(modelo_produccion, data=df_bogota)
predicciones = predicciones.rename(columns={'prediction_label': 'IVC_predicho'})
display(predicciones[['IVC_predicho']].describe())
```

Descargar y leer el shapefile

In [32]:
```
import geopandas as gpd
import zipfile
import os

wr.s3.download('s3://bogotamanzanas/manzana.0225.shp.zip', local_file='manzana.0225.shp.zip')

with zipfile.ZipFile('manzana.0225.shp.zip', 'r') as z:
    z.extractall('manzana_shp')

shp_files = [f for f in os.listdir('manzana_shp') if f.endswith('.shp')]
print(shp_files)

gdf_manzanas = gpd.read_file(f'manzana_shp/{shp_files[0]}')
print(gdf_manzanas.shape)
print(gdf_manzanas.columns.tolist())
gdf_manzanas.head()
```

```
['MANZANA.shp']
(44057, 5)
['MANCODIGO', 'SECCODIGO', 'SHAPE_Leng', 'SHAPE_Area', 'geometry']
```

Out[32]:

	MANCODIGO	SECCODIGO	SHAPE_Leng	SHAPE_Area	geometry
0	004501049	004501	0.004340	6.849815e-07	POLYGON ((-74.13655 4.62794, -74.13656 4.62785...
1	004537005	004537	0.002015	2.452513e-07	POLYGON ((-74.17189 4.61561, -74.17174 4.6154,...
2	002569081	002569	0.000975	4.103821e-08	POLYGON ((-74.16526 4.56347, -74.16527 4.56344...
3	004537006	004537	0.002056	2.603283e-07	POLYGON ((-74.17212 4.61557, -74.17233 4.6154,...
4	004537009	004537	0.002006	2.393863e-07	POLYGON ((-74.17146 4.61533, -74.17147 4.61532...

Comparar para llave comun

In [33]:
```
print("Columnas CSV (predicciones):", predicciones.columns.tolist())
print("\nColumnas Shapefile:", gdf_manzanas.columns.tolist())
```

```
Columnas CSV (predicciones): ['CD_LC_CM', 'NMB_LC_CM', 'TP_LC_CM', 'AREA', 'LATITUD', 'LONGITUD', 'Shape_Leng', 'Shape_Area', 'TP27
_PERSO', 'TVIVIENDA', 'TP16_HOG', 'DENSIDAD', 'TP34_1_EDA', 'TP34_8_EDA', 'TP34_9_EDA', 'TP51_13_ED', 'TP51POSTGR', 'TP51PRIMAR',
'TP51SECUND', 'TP51SUPERI', 'TP51_99_ED', 'TP14_6_TIP', 'TP15_4_OCU', 'TP9_3_10_N', 'TP9_3_6_NO', 'TP14_3_TIP', 'TP15_1_OCU', 'TP15
_2_OCU', 'TP15_3_OCU', 'TP9_3_8_NO', 'TP9_3_9_NO', 'TP9_2_4_MI', 'TP9_3_4_NO', 'TP19_EE_2', 'TP19_ACU_2', 'TP19_ALC_2', 'TP19_GAS_
2', 'TP19_RECB2', 'TP19_INTE2', 'TP19_GAS_9', 'TP19_INTE9', 'TP19_EE_1', 'TP19_ACU_1', 'TP19_ALC_1', 'TP19_GAS_1', 'TP19_RECB1', 'T
P19_INTE1', 'TP19_EE_E1', 'TP19_EE_E2', 'TP19_EE_E3', 'TP19_EE_E4', 'TP19_EE_E5', 'TP19_EE_E6', 'TP19_EE_E9', 'TP9_1_USO', 'TP9_2_U
SO', 'TP9_3_USO', 'ppt_mean_2000_2018', 'ivc_formula', 'categoria_ivc', 'perfil_vulnerabilidad', 'ivc_n', 'IVC_predicho']

Columnas Shapefile: ['MANCODIGO', 'SECCODIGO', 'SHAPE_Leng', 'SHAPE_Area', 'geometry']
```

In [34]:

```
print("Muestra CD_LC_CM (CSV):")
print(predicciones['CD_LC_CM'].head(10))
print("Tipo de dato:", predicciones['CD_LC_CM'].dtype)

print("\nMuestra MANCODIGO (Shapefile):")
print(gdf_manzanas['MANCODIGO'].head(10))
print("Tipo de dato:", gdf_manzanas['MANCODIGO'].dtype)

print("\nMuestra SECCODIGO (Shapefile):")
print(gdf_manzanas['SECCODIGO'].head(10))
```

```
Muestra CD_LC_CM (CSV):
0    4.0
1    4.0
2    4.0
3    4.0
4    4.0
5    4.0
6    4.0
7    4.0
8    4.0
9    4.0
Name: CD_LC_CM, dtype: float32
Tipo de dato: float32

Muestra MANCODIGO (Shapefile):
0    004501049
1    004537005
2    002569081
3    004537006
4    004537009
5    004537008
6    004545016
7    007207010
8    009130011
9    002555083
Name: MANCODIGO, dtype: object
Tipo de dato: object

Muestra SECCODIGO (Shapefile):
0    004501
1    004537
2    002569
3    004537
4    004537
5    004537
6    004545
7    007207
8    009130
9    002555
Name: SECCODIGO, dtype: object
```

In [35]:

```python
import geopandas as gpd
from shapely.geometry import Point

# Crear geometría de puntos a partir de LATITUD/LONGITUD
geometry_puntos = [Point(xy) for xy in zip(predicciones['LONGITUD'], predicciones['LATITUD'])]

gdf_predicciones = gpd.GeoDataFrame(
```

```
    predicciones,
    geometry=geometry_puntos,
    crs='EPSG:4326'  # WGS84, sistema estándar de lat/long
)

print(gdf_predicciones.crs)
print(gdf_manzanas.crs)
```

```
EPSG:4326
EPSG:4686
```

Paso 1 — Alinear el sistema de coordenadas

In [36]:
```
gdf_predicciones = gdf_predicciones.to_crs(gdf_manzanas.crs)

print("CRS predicciones:", gdf_predicciones.crs)
print("CRS shapefile:", gdf_manzanas.crs)
print("¿Coinciden?", gdf_predicciones.crs == gdf_manzanas.crs)
```

```
CRS predicciones: EPSG:4686
CRS shapefile: EPSG:4686
¿Coinciden? True
```

Paso 2 — Cruce espacial (point-in-polygon)

In [37]:
```
gdf_final = gpd.sjoin(
    gdf_predicciones,
    gdf_manzanas[['MANCODIGO', 'SECCODIGO', 'geometry']],
    how='left',
    predicate='within'
)

print("Shape resultado:", gdf_final.shape)
print("Sin manzana asignada:", gdf_final['MANCODIGO'].isna().sum())
print("Con manzana asignada:", gdf_final['MANCODIGO'].notna().sum())
display(gdf_final[['MANCODIGO', 'SECCODIGO', 'LATITUD', 'LONGITUD', 'IVC_predicho']].head(10))
```

```
Shape resultado: (43422, 67)
Sin manzana asignada: 1421
Con manzana asignada: 42001
```

	MANCODIGO	SECCODIGO	LATITUD	LONGITUD	IVC_predicho
0	001101001	001101	4.585633	-74.081619	0.168510
1	001101002	001101	4.584788	-74.080811	0.217634
2	001101006	001101	4.582480	-74.081047	0.129159
3	001101009	001101	4.582606	-74.083260	0.138875
4	001101008	001101	4.582664	-74.079529	0.205890
5	001101005	001101	4.583165	-74.081802	0.211296
6	001101003	001101	4.584102	-74.081284	0.209833
7	001101004	001101	4.583639	-74.081589	0.199431
8	001102046	001102	4.581050	-74.081734	0.148708
9	001102058	001102	4.581325	-74.082283	0.213984

Paso 3 — Validar el cruce (obligatorio antes de exportar)

In [38]:
```
# Si hay puntos sin asignar, prueba con 'intersects' (más tolerante en bordes)
if gdf_final['MANCODIGO'].isna().sum() > 0:
    gdf_final_v2 = gpd.sjoin(
        gdf_predicciones, gdf_manzanas[['MANCODIGO', 'SECCODIGO', 'geometry']],
        how='left', predicate='intersects'
    )
    print("Sin asignar (intersects):", gdf_final_v2['MANCODIGO'].isna().sum())
    if gdf_final_v2['MANCODIGO'].isna().sum() < gdf_final['MANCODIGO'].isna().sum():
        gdf_final = gdf_final_v2
        print("Se usó 'intersects' por mejor cobertura")
```

```
Sin asignar (intersects): 1421
```

Paso 4 — Revisar duplicados (un punto puede caer en 2 polígonos si comparten borde)

In [39]:
```
duplicados = gdf_final.index.duplicated().sum()
print("Filas duplicadas por el join:", duplicados)

if duplicados > 0:
    gdf_final = gdf_final[~gdf_final.index.duplicated(keep='first')]
    print("Duplicados eliminados. Shape final:", gdf_final.shape)
```

```
Filas duplicadas por el join: 14
Duplicados eliminados. Shape final: (43408, 67)
```

Paso 5 — Limpiar columnas antes de exportar

In [40]:
```
gdf_final_export = gdf_final.drop(columns=['index_right'], errors='ignore')

# Verifica que las columnas clave estén presentes
print(gdf_final_export[['MANCODIGO', 'SECCODIGO', 'IVC_predicho']].isna().sum())
```

```
MANCODIGO       1421
SECCODIGO       1421
IVC_predicho       0
dtype: int64
```

Paso 6 — Guardar el shapefile localmente

In [41]:
```
gdf_final_export.to_file('manzanas_ivc_predicho.shp')
print("Shapefile guardado localmente")
```

```
Shapefile guardado localmente
```

Paso 7 — Comprimir en .zip (formato estándar de distribución de shapefiles)

In [42]:
```
import zipfile, glob

with zipfile.ZipFile('manzanas_ivc_predicho.zip', 'w') as z:
    for f in glob.glob('manzanas_ivc_predicho.*'):
        z.write(f)

print("Archivos comprimidos:", glob.glob('manzanas_ivc_predicho.*'))
```

```
Archivos comprimidos: ['manzanas_ivc_predicho.cpg', 'manzanas_ivc_predicho.dbf', 'manzanas_ivc_predicho.prj', 'manzanas_ivc_predich
o.shp', 'manzanas_ivc_predicho.shx', 'manzanas_ivc_predicho.zip']
```

Paso 8 — Subir a S3

In [43]:
```
wr.s3.upload('manzanas_ivc_predicho.zip', 's3://bogotamanzanas/manzanas_ivc_predicho.zip')
print("Subido a: s3://bogotamanzanas/manzanas_ivc_predicho.zip")
```

```
Subido a: s3://bogotamanzanas/manzanas_ivc_predicho.zip
```

Paso 9 — Verificación final (confirmar que quedó en S3

In [44]:
```
archivos = wr.s3.list_objects('s3://bogotamanzanas/')
print(archivos)
```

```
['s3://bogotamanzanas/bogota_ivc_thesis_ready.csv', 's3://bogotamanzanas/manzana.0225.shp.zip', 's3://bogotamanzanas/manzanas_ivc_p
redicho.zip']
```

10.Mapa

In [51]:
```
# Dissolve por MANCODIGO, promediando solo la columna numérica
gdf_manzanas_promedio = gdf_mapa[['MANCODIGO', 'IVC_predicho', 'geometry']].dissolve(
    by='MANCODIGO',
    aggfunc={'IVC_predicho': 'mean'}
).reset_index()

print(gdf_manzanas_promedio.shape)
display(gdf_manzanas_promedio.head())
```

```
(44023, 3)
```

	MANCODIGO	geometry	IVC_predicho
0	001101001	POLYGON ((999536.097 998918.702, 999536.167 99...	0.168510
1	001101002	POLYGON ((999622.402 998834.001, 999626.024 99...	0.217634
2	001101003	POLYGON ((999669.167 998630.4, 999671.925 9986...	0.209833
3	001101004	POLYGON ((999503.931 998643.764, 999507.392 99...	0.199431
4	001101005	POLYGON ((999583.691 998567.142, 999577.677 99...	0.211296

In [52]:
```
gdf_manzanas_promedio = gdf_mapa[['MANCODIGO', 'NMB_LC_CM', 'IVC_predicho', 'geometry']].dissolve(
    by='MANCODIGO',
    aggfunc={
        'IVC_predicho': 'mean',
        'NMB_LC_CM': 'first'   # conserva el nombre de localidad asociado a esa manzana
    }
).reset_index()

print(gdf_manzanas_promedio.shape)
display(gdf_manzanas_promedio.head())
```

```
(44023, 4)
```

	MANCODIGO	geometry	IVC_predicho	NMB_LC_CM
0	001101001	POLYGON ((999536.097 998918.702, 999536.167 99...	0.168510	SAN CRISTOBAL
1	001101002	POLYGON ((999622.402 998834.001, 999626.024 99...	0.217634	SAN CRISTOBAL
2	001101003	POLYGON ((999669.167 998630.4, 999671.925 9986...	0.209833	SAN CRISTOBAL
3	001101004	POLYGON ((999503.931 998643.764, 999507.392 99...	0.199431	SAN CRISTOBAL
4	001101005	POLYGON ((999583.691 998567.142, 999577.677 99...	0.211296	SAN CRISTOBAL

In [53]:
```
print("Manzanas originales en shapefile:", gdf_manzanas['MANCODIGO'].nunique())
print("Manzanas después del dissolve:", gdf_manzanas_promedio.shape[0])
print("Nulos en IVC_predicho:", gdf_manzanas_promedio['IVC_predicho'].isna().sum())
```

```
Manzanas originales en shapefile: 44023
Manzanas después del dissolve: 44023
Nulos en IVC_predicho: 4365
```

In [54]:
```
gdf_mapa_final = gdf_manzanas_promedio[gdf_manzanas_promedio['IVC_predicho'].notna()].copy()

print(f"Manzanas con dato: {gdf_mapa_final.shape[0]}")
print(f"Manzanas excluidas (sin dato): {44023 - gdf_mapa_final.shape[0]}")
```

```
Manzanas con dato: 39658
Manzanas excluidas (sin dato): 4365
```

In [73]:
```
gdf_mapa_final = gdf_mapa_final.to_crs(epsg=3116)  # MAGNA-SIRGAS / Bogotá, en metros
print(gdf_mapa_final.crs)
```

```
EPSG:3116
```

In [81]:
```
%pip install contextily matplotlib-scalebar
```

```
Collecting contextily
  Using cached contextily-1.7.1-py3-none-any.whl.metadata (3.1 kB)
Requirement already satisfied: matplotlib-scalebar in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (0.
9.0)
Collecting geopy (from contextily)
  Using cached geopy-2.5.0-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: matplotlib in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from contex
tily) (3.10.9)
Collecting mercantile (from contextily)
  Using cached mercantile-1.2.1-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: pillow in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from contextil
y) (12.3.0)
Collecting rasterio (from contextily)
  Downloading rasterio-1.4.4-cp310-cp310-manylinux_2_28_x86_64.whl.metadata (9.3 kB)
Requirement already satisfied: requests in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from contexti
ly) (2.34.2)
Requirement already satisfied: joblib in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from contextil
y) (1.3.2)
Requirement already satisfied: xyzservices in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from conte
xtily) (2026.3.0)
Collecting geographiclib<3,>=1.52 (from geopy->contextily)
  Using cached geographiclib-2.1-py3-none-any.whl.metadata (1.6 kB)
Requirement already satisfied: contourpy>=1.0.1 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from
matplotlib->contextily) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from matp
lotlib->contextily) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from
matplotlib->contextily) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from
matplotlib->contextily) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from matpl
otlib->contextily) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from m
atplotlib->contextily) (26.2)
Requirement already satisfied: pyparsing>=3 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from matp
lotlib->contextily) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (f
rom matplotlib->contextily) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from python-d
ateutil>=2.7->matplotlib->contextily) (1.17.0)
Requirement already satisfied: click>=3.0 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from mercan
tile->contextily) (8.4.2)
Collecting affine (from rasterio->contextily)
  Using cached affine-3.0.0-py3-none-any.whl.metadata (4.3 kB)
Requirement already satisfied: attrs in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from rasterio->c
ontextily) (26.1.0)
Requirement already satisfied: certifi in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from rasterio-
>contextily) (2026.7.22)
```

```
Collecting cligj>=0.5 (from rasterio->contextily)
  Using cached cligj-0.7.2-py3-none-any.whl.metadata (5.0 kB)
Collecting click-plugins (from rasterio->contextily)
  Using cached click_plugins-1.1.1.2-py2.py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-package
s (from requests->contextily) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from requ
ests->contextily) (3.18)
Requirement already satisfied: urllib3<3,>=1.26 in /home/sagemaker-user/.conda/envs/pycaret_env/lib/python3.10/site-packages (from
requests->contextily) (2.7.0)
Using cached contextily-1.7.1-py3-none-any.whl (18 kB)
Using cached geopy-2.5.0-py3-none-any.whl (114 kB)
Using cached geographiclib-2.1-py3-none-any.whl (40 kB)
Using cached mercantile-1.2.1-py3-none-any.whl (14 kB)
Downloading rasterio-1.4.4-cp310-cp310-manylinux_2_28_x86_64.whl (35.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 35.3/35.3 MB 109.2 MB/s  0:00:00eta 0:00:01
Using cached cligj-0.7.2-py3-none-any.whl (7.1 kB)
Using cached affine-3.0.0-py3-none-any.whl (10 kB)
Using cached click_plugins-1.1.1.2-py2.py3-none-any.whl (11 kB)
Installing collected packages: mercantile, geographiclib, cligj, click-plugins, affine, rasterio, geopy, contextily
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8/8 [contextily]8 [geopy]io]
Successfully installed affine-3.0.0 click-plugins-1.1.1.2 cligj-0.7.2 contextily-1.7.1 geographiclib-2.1 geopy-2.5.0 mercantile-1.
2.1 rasterio-1.4.4
Note: you may need to restart the kernel to use updated packages.
```

In [88]:

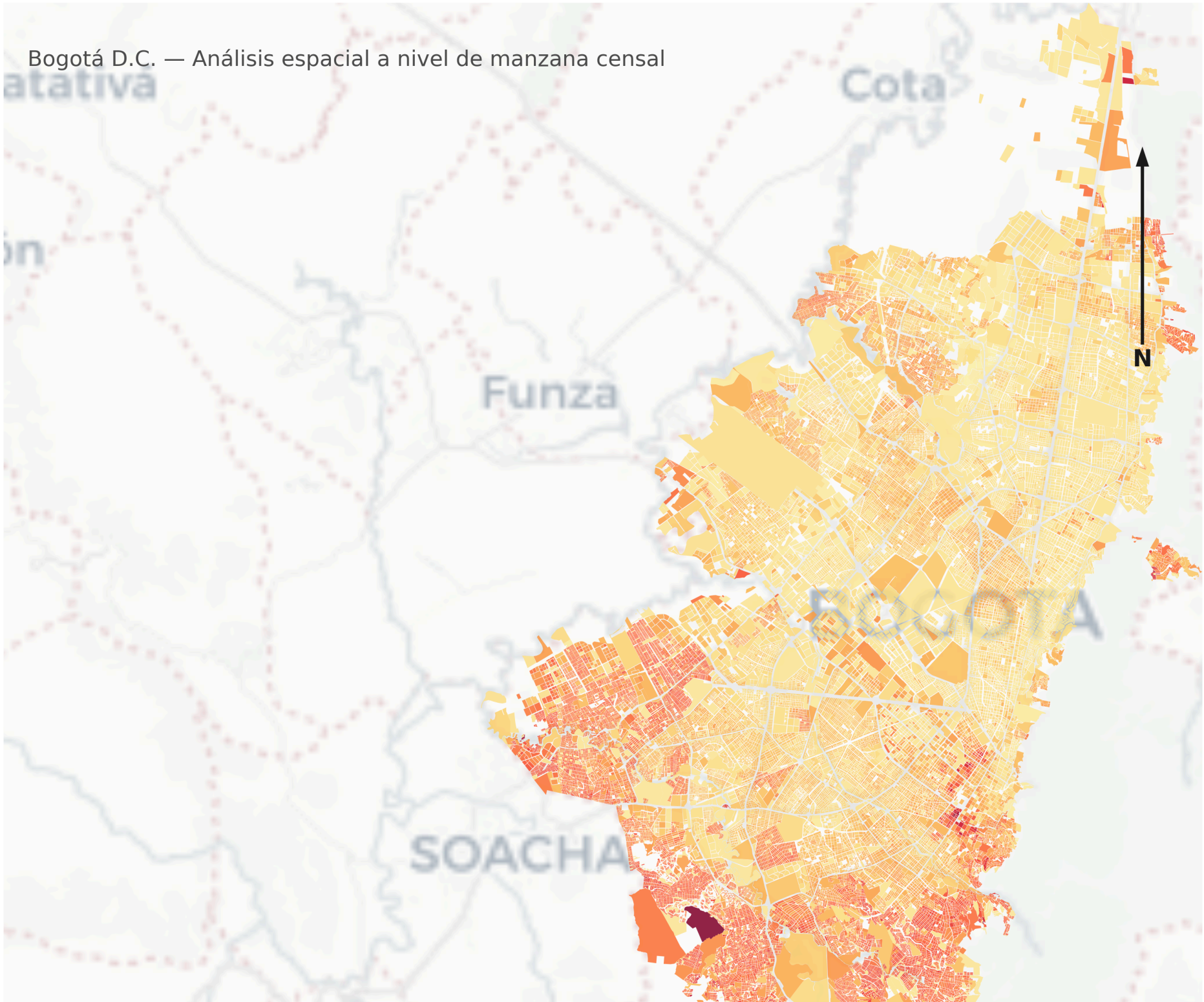
```python
import matplotlib.pyplot as plt
import contextily as ctx
from matplotlib_scalebar.scalebar import ScaleBar

# --- 1. REPROYECCIÓN (Obligatorio para alinear perfecto con mapas web) ---
gdf_mapa_plot = gdf_mapa_final.to_crs(epsg=3857)

# --- 2. MATEMÁTICA DEL ENCUADRE (El secreto para cero espacio blanco) ---
minx, miny, maxx, maxy = gdf_mapa_plot.total_bounds
ancho_mapa = maxx - minx
alto_mapa = maxy - miny
aspecto_mapa = alto_mapa / ancho_mapa

# Definimos el ancho deseado en pulgadas (ej. 10 pulgadas)
ancho_figura = 10.0
# Calculamos el alto EXACTO que necesita el mapa para no deformarse
alto_mapa_figura = ancho_figura * aspecto_mapa

# Añadimos espacio fijo en pulgadas para el título (arriba) y metadatos (abajo)
margen_superior = 1.5
margen_inferior = 1.5
alto_total_figura = margen_superior + alto_mapa_figura + margen_inferior
```

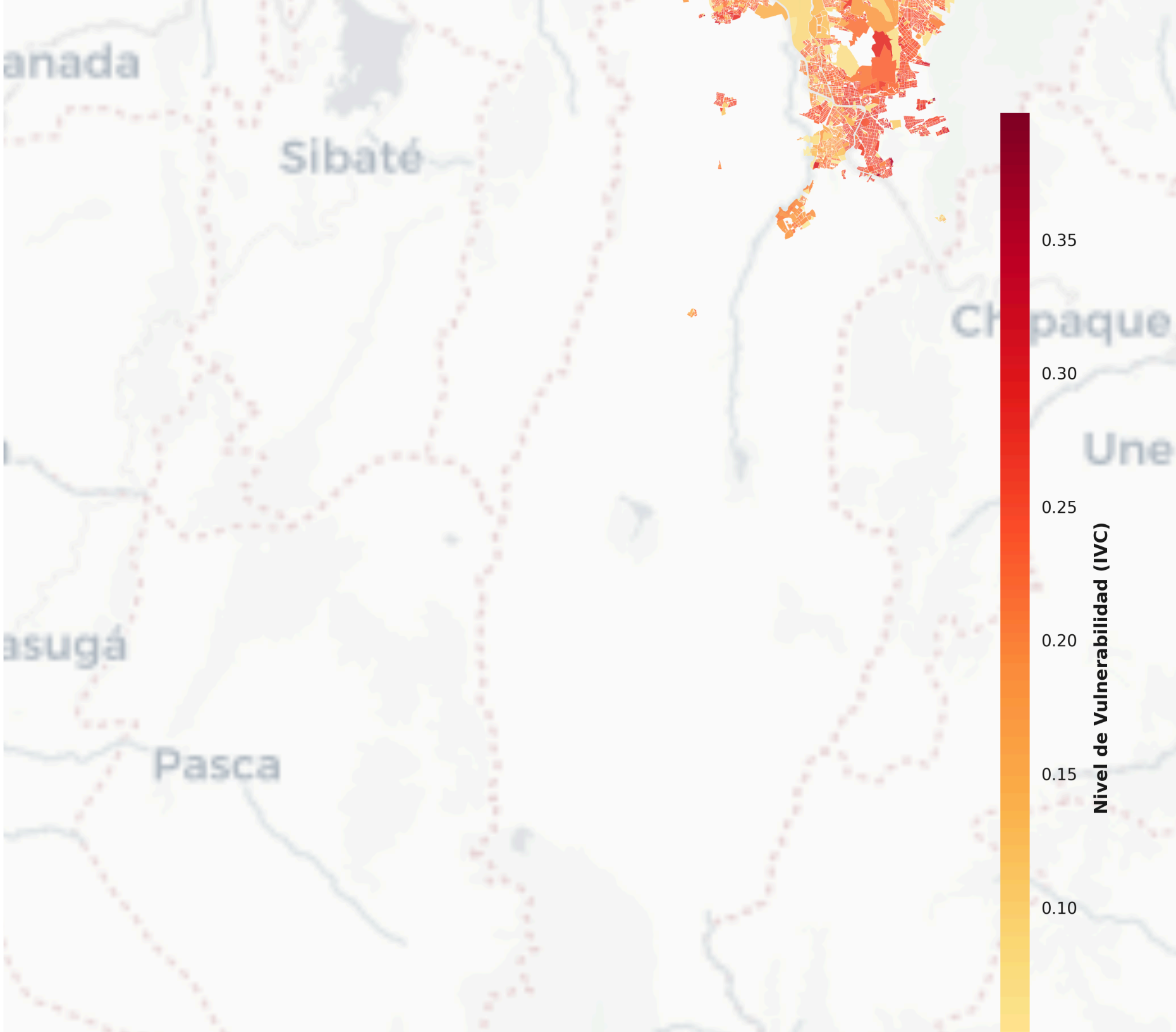
```python
# Creamos el lienzo con estas dimensiones exactas
fig = plt.figure(figsize=(ancho_figura, alto_total_figura), dpi=300)
fig.patch.set_facecolor('white')

# Posicionamos los ejes del mapa usando fracciones (0 a 1)
# [izq, abajo, ancho, alto]. El ancho es 1.0 (100%), no sobra nada a los lados.
bottom_relativo = margen_inferior / alto_total_figura
alto_relativo = alto_mapa_figura / alto_total_figura
ax = fig.add_axes([0, bottom_relativo, 1.0, alto_relativo])
ax.set_facecolor('white')

# --- 3. RENDERIZADO DEL MAPA (Colores Nivel Maestría) ---
gdf_mapa_plot.plot(
    column='IVC_predicho',
    cmap='YlOrRd',          # Estándar académico para Vulnerabilidad
    linewidth=0,
    edgecolor='none',
    alpha=0.85,
    ax=ax,
    legend=False,
    missing_kwds={"color": "none"}
)

# --- 4. MAPA BASE ---
# Positron es un mapa en escala de grises muy limpio que hace resaltar los datos
ctx.add_basemap(ax, source=ctx.providers.CartoDB.Positron, alpha=0.7)

# Fijamos los límites de manera absoluta
ax.set_xlim(minx, maxx)
ax.set_ylim(miny, maxy)
ax.set_aspect('equal')
ax.axis('off') # Apagamos el marco del gráfico

# --- 5. TÍTULOS E INFORMACIÓN ACADÉMICA ---
# Título arriba (coordenadas de la figura completa)
fig.text(0.02, 0.95, "Índice de Vulnerabilidad Climática (IVC) Predicho",
         fontsize=18, fontweight='bold', color='#1A1A1A', family='sans-serif')
fig.text(0.02, 0.925, "Bogotá D.C. — Análisis espacial a nivel de manzana censal",
         fontsize=13, color='#4D4D4D', family='sans-serif')

# --- 6. BARRA DE COLOR FLOTANTE ---
vmin, vmax = gdf_mapa_plot['IVC_predicho'].min(), gdf_mapa_plot['IVC_predicho'].max()
sm = plt.cm.ScalarMappable(cmap='YlOrRd', norm=plt.Normalize(vmin=vmin, vmax=vmax))
sm._A = []

# Posicionamos la leyenda dentro del área del mapa (abajo a la izquierda/derecha) para ahorrar espacio
cax = fig.add_axes([0.85, 0.25, 0.025, 0.35])
cb = fig.colorbar(sm, cax=cax)
```

```python
cb.set_label("Nivel de Vulnerabilidad (IVC)", fontsize=11, labelpad=10, fontweight='bold', color='#1A1A1A')
cb.outline.set_visible(False)
cb.ax.tick_params(size=0, labelsize=10, colors='#1A1A1A')

# --- 7. NORTE Y ESCALA ---
ax.annotate('N', xy=(0.95, 0.95), xytext=(0.95, 0.88),
            xycoords='axes fraction',
            arrowprops=dict(facecolor='#1A1A1A', edgecolor='none', width=2, headwidth=8, headlength=12),
            ha='center', va='top', fontsize=14, fontweight='bold', color='#1A1A1A')

escala = ScaleBar(
    dx=1, units="m", location='lower right',
    box_alpha=0.7, box_color='white', color='#1A1A1A',
    font_properties={'size': 10, 'weight': 'bold'}, border_pad=1
)
ax.add_artist(escala)

# --- 8. METADATOS DEL MODELO DE MACHINE LEARNING (FOOTER) ---
texto_metadatos = (
    "NOTAS METODOLÓGICAS:\n"
    "• Datos base: Variables climáticas satelitales (2000-2018) y microdatos censales (DANE).\n"
    f"• Algoritmo Predictivo: Light Gradient Boosting Machine (LightGBM). Precisión del modelo: R² = 0.899.\n"
    f"• Universo de estudio: N = {gdf_mapa_plot.shape[0]:,} manzanas censales (valores atípicos excluidos mediante rango intercuar
    "• Sistema de Referencia de Coordenadas: Proyectado a Web Mercator (EPSG:3857) para cartografía base."
)
fig.text(0.02, 0.03, texto_metadatos, fontsize=9, color='#4D4D4D', ha='left', linespacing=1.6)

# --- 9. GUARDAR CON RECORTE ABSOLUTO ---
plt.savefig('mapa_tesis_maestria_optimo.png',
            dpi=300,
            bbox_inches='tight', # Corta cualquier pixel sobrante
            pad_inches=0.05,     # Borde blanco mínimo de respiración
            facecolor='white')
plt.show()
```

Índice de Vulnerabilidad Climática (IVC) Predicho

Bogotá D.C. — Análisis espacial a nivel de manzana censal




anada
Sibaté
Chipaque
Une
asugá
Pasca
0.35
0.30
0.25
0.20
0.15
0.10
Nivel de Vulnerabilidad (IVC)

NOTAS METODOLÓGICAS:

- Datos base: Variables climáticas satelitales (2000-2018) y microdatos censales (DANE).
- Algoritmo Predictivo: Light Gradient Boosting Machine (LightGBM). Precisión del modelo: $R^2 = 0.899$.
- Universo de estudio: N = 39,658 manzanas censales (valores atípicos excluidos mediante rango intercuartílico - IQR).
- Sistema de Referencia de Coordenadas: Proyectado a Web Mercator (EPSG:3857) para cartografía base.

In [98]:

```
import boto3

# --- 1. CONFIGURACIÓN ---
bucket_name = 'prediccionesbogota'
```

```python
# Nombres de los archivos locales
nombre_mapa_png = 'mapa_tesis_maestria_optimo.png'
nombre_datos_csv = 'datos_ivc_manzanas_bogota.csv'

# --- 2. CREAR Y GUARDAR EL CSV LOCALMENTE ---
print(f"Generando el archivo {nombre_datos_csv}...")
# Descartamos 'geometry' para que el CSV sea tabular, ligero y fácil de leer
gdf_mapa_final.drop(columns=['geometry']).to_csv(nombre_datos_csv, index=False, encoding='utf-8')
print("✅ Archivo CSV generado correctamente en el entorno local.")

# --- 3. CONEXIÓN A AWS S3 Y SUBIDA DE ARCHIVOS ---
s3 = boto3.client('s3')

print(f"\nIniciando subida al bucket '{bucket_name}'...")

try:
    # Subir el mapa PNG (lo guardamos dentro de una carpeta 'mapas/')
    ruta_s3_png = f'mapas/{nombre_mapa_png}'
    s3.upload_file(nombre_mapa_png, bucket_name, ruta_s3_png)
    print(f"✅ Mapa subido exitosamente a: s3://{bucket_name}/{ruta_s3_png}")

    # Subir el archivo CSV (lo guardamos dentro de una carpeta 'datos/')
    ruta_s3_csv = f'datos/{nombre_datos_csv}'
    s3.upload_file(nombre_datos_csv, bucket_name, ruta_s3_csv)
    print(f"✅ Datos CSV subidos exitosamente a: s3://{bucket_name}/{ruta_s3_csv}")

    print("\n🚀 ¡Proceso finalizado! Todos los archivos de tu maestría están respaldados en S3.")

except FileNotFoundError as e:
    print(f"\n❌ Error: No se encontró el archivo local. Asegúrate de haber ejecutado la celda que crea el mapa antes de correr es
except Exception as e:
    print(f"\n❌ Error al intentar subir a S3: {e}")
    print("💡 Tip: Verifica que el rol de IAM asociado a esta instancia de SageMaker tenga el permiso 's3:PutObject' para el bucke
```

```
Generando el archivo datos_ivc_manzanas_bogota.csv...
✅ Archivo CSV generado correctamente en el entorno local.

Iniciando subida al bucket 'prediccionesbogota'...
✅ Mapa subido exitosamente a: s3://prediccionesbogota/mapas/mapa_tesis_maestria_optimo.png
✅ Datos CSV subidos exitosamente a: s3://prediccionesbogota/datos/datos_ivc_manzanas_bogota.csv

🚀 ¡Proceso finalizado! Todos los archivos de tu maestría están respaldados en S3.
```

In [100…

```python
import matplotlib.pyplot as plt
import contextily as ctx
from matplotlib_scalebar.scalebar import ScaleBar
```

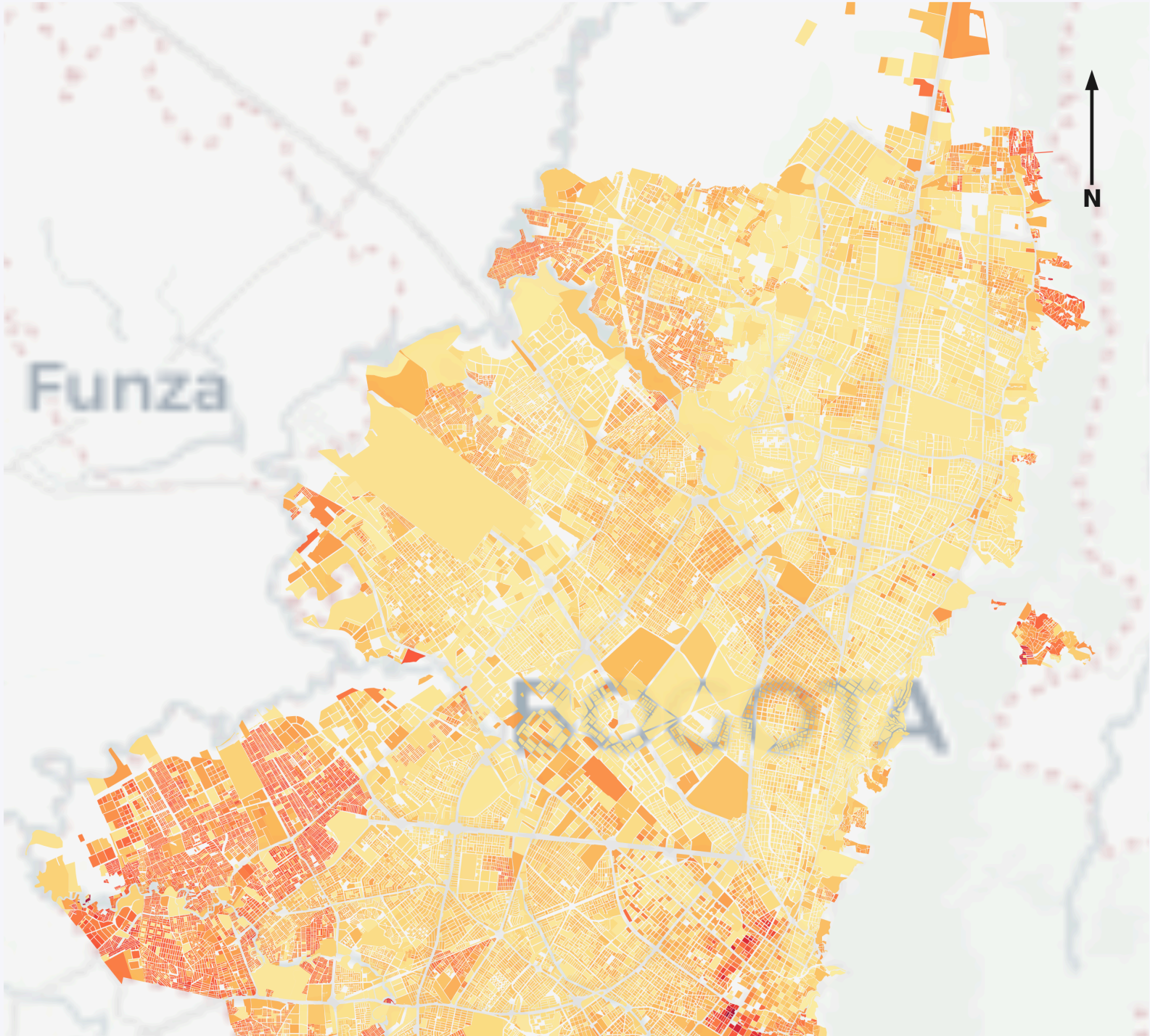
```python
# --- 1. REPROYECCIÓN A WEB MERCATOR ---
gdf_mapa_plot = gdf_mapa_final.to_crs(epsg=3857)

# --- 2. CORRECCIÓN DEL ENCUADRE (Filtrando outliers espaciales) ---
# En lugar de usar total_bounds (que toma todo), calculamos los límites
# basados en el 99% de tus datos. Esto recorta las manzanas aisladas/lejanas
# y hace un zoom perfecto a la zona densa de Bogotá.
limites = gdf_mapa_plot.bounds
minx = limites['minx'].quantile(0.005) # Corta el 0.5% más a la izquierda
maxx = limites['maxx'].quantile(0.995) # Corta el 0.5% más a la derecha
miny = limites['miny'].quantile(0.01)  # Corta el 1% más al sur
maxy = limites['maxy'].quantile(0.99)  # Corta el 1% más al norte

# AUMENTO AL 12%: Añadimos el margen (buffer) del 12% para que no corte los bordes
buffer_x = (maxx - minx) * 0.12
buffer_y = (maxy - miny) * 0.12
minx, maxx = minx - buffer_x, maxx + buffer_x
miny, maxy = miny - buffer_y, maxy + buffer_y

# --- 3. CÁLCULO DE PROPORCIONES DEL LIENZO ---
ancho_mapa = maxx - minx
alto_mapa = maxy - miny
aspecto_mapa = alto_mapa / ancho_mapa

ancho_figura = 12
alto_figura = ancho_figura * aspecto_mapa

# Creamos la figura
fig, ax = plt.subplots(figsize=(ancho_figura, alto_figura), dpi=300)
fondo_color = '#F8F9FA' # Un gris perla ultra claro, muy elegante
fig.patch.set_facecolor(fondo_color)
ax.set_facecolor(fondo_color)

# --- 4. RENDERIZADO DEL MAPA ---
gdf_mapa_plot.plot(
    column='IVC_predicho',
    cmap='YlOrRd',
    linewidth=0,
    edgecolor='none',
    alpha=0.9,
    ax=ax,
    legend=False,
    missing_kwds={"color": "none"}
)

# --- 5. MAPA BASE ---
ctx.add_basemap(ax, source=ctx.providers.CartoDB.Positron, alpha=0.8)
```

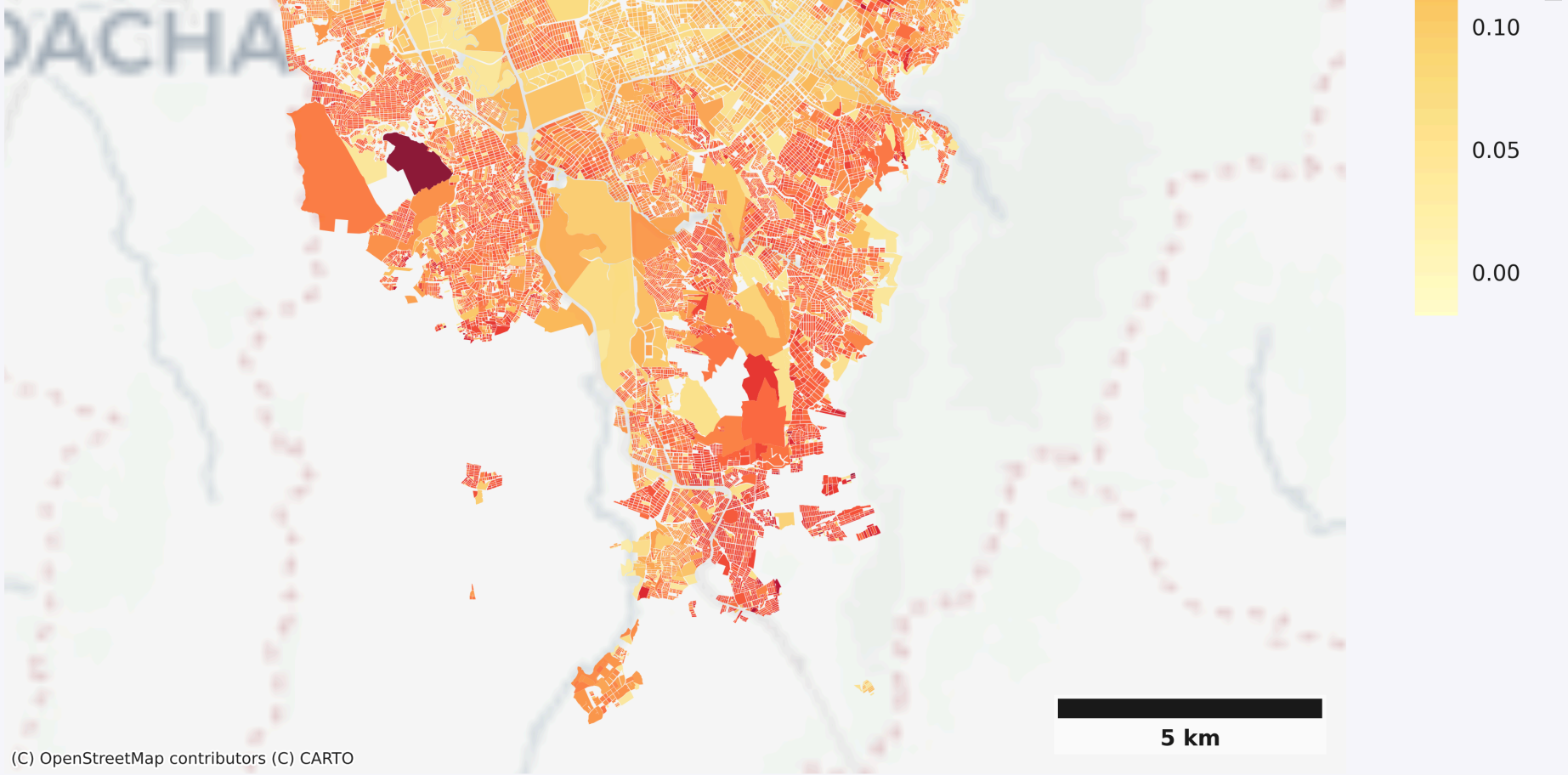
```python
# Aplicamos los límites calculados sin deformar el mapa
ax.set_xlim(minx, maxx)
ax.set_ylim(miny, maxy)
ax.set_aspect('equal')
ax.set_axis_off()

# Ajustamos los márgenes internos de la figura para dejar espacio ordenado
plt.subplots_adjust(top=0.90, bottom=0.12, left=0.02, right=0.82)

# --- 6. TÍTULOS E INFORMACIÓN ---
fig.text(0.02, 0.95, "Índice de Vulnerabilidad Climática (IVC) Predicho",
         fontsize=20, fontweight='bold', color='#1A1A1A')
fig.text(0.02, 0.92, "Bogotá D.C. — Análisis espacial a nivel de manzana censal",
         fontsize=14, color='#4D4D4D')

# --- 7. BARRA DE COLOR (Mejor Proporcionada) ---
vmin, vmax = gdf_mapa_plot['IVC_predicho'].min(), gdf_mapa_plot['IVC_predicho'].max()
sm = plt.cm.ScalarMappable(cmap='YlOrRd', norm=plt.Normalize(vmin=vmin, vmax=vmax))
sm._A = []

# La hacemos más ancha y menos larga para que no se vea como una línea delgada
cax = fig.add_axes([0.85, 0.30, 0.025, 0.40])
cb = fig.colorbar(sm, cax=cax)
cb.set_label("Nivel de Vulnerabilidad (IVC)", fontsize=12, labelpad=12, fontweight='bold', color='#1A1A1A')
cb.outline.set_visible(False)
cb.ax.tick_params(size=0, labelsize=11, colors='#1A1A1A')

# --- 8. NORTE Y ESCALA GRÁFICA ---
ax.annotate('N', xy=(0.95, 0.96), xytext=(0.95, 0.89),
            xycoords='axes fraction',
            arrowprops=dict(facecolor='#1A1A1A', edgecolor='none', width=2, headwidth=8, headlength=12),
            ha='center', va='top', fontsize=14, fontweight='bold', color='#1A1A1A')

escala = ScaleBar(
    dx=1, units="m", location='lower right',
    box_alpha=0.7, box_color='white', color='#1A1A1A',
    font_properties={'size': 11, 'weight': 'bold'}, border_pad=1
)
ax.add_artist(escala)

# --- 9. METADATOS DEL MODELO (FOOTER ORDENADO) ---
texto_metadatos = (
    "NOTAS METODOLÓGICAS:\n"
    "• Datos base: Variables climáticas satelitales (2000-2018) y microdatos censales (DANE).\n"
    f"• Algoritmo Predictivo: LightGBM (R² = 0.899). Universo analizado: {gdf_mapa_plot.shape[0]:,} manzanas censales.\n"
    "• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR)."
)
# Alineado abajo a la izquierda de forma limpia
```

```
fig.text(0.02, 0.02, texto_metadatos, fontsize=10, color='#4D4D4D', ha='left', linespacing=1.6)

# --- 10. GUARDAR MAPA CORREGIDO ---
plt.savefig('mapa_tesis_maestria_corregido.png',
            dpi=300,
            bbox_inches='tight',
            pad_inches=0.1,
            facecolor=fondo_color)
plt.show()
```

Índice de Vulnerabilidad Climática (IVC) Predicho

Bogotá D.C. — Análisis espacial a nivel de manzana censal





NOTAS METODOLÓGICAS:
• Datos base: Variables climáticas satelitales (2000-2018) y microdatos censales (DANE).
• Algoritmo Predictivo: LightGBM ($R^2 = 0.899$). Universo analizado: 39,658 manzanas censales.
• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR).

In [ ]:
```
import boto3

# --- 1. CONFIGURACIÓN ---
bucket_name = 'prediccionesbogota'

# Nombres de los archivos locales (el PNG que acabas de generar y el CSV)
nombre_mapa_png = 'mapa_tesis_maestria_corregido.png'
nombre_datos_csv = 'datos_ivc_manzanas_bogota.csv'

# --- 2. CREAR Y GUARDAR EL CSV LOCALMENTE ---
```

```
print(f"Generando el archivo {nombre_datos_csv}...")
# Descartamos 'geometry' para que el CSV sea tabular, ligero y fácil de leer
gdf_mapa_final.drop(columns=['geometry']).to_csv(nombre_datos_csv, index=False, encoding='utf-8')
print("✅ Archivo CSV generado correctamente en el entorno local.")

# --- 3. CONEXIÓN A AWS S3 Y SUBIDA DE ARCHIVOS ---
s3 = boto3.client('s3')

print(f"\nIniciando subida al bucket '{bucket_name}'...")

try:
    # Subir el mapa PNG (lo guardamos dentro de una carpeta 'mapas/')
    ruta_s3_png = f'mapas/{nombre_mapa_png}'
    s3.upload_file(nombre_mapa_png, bucket_name, ruta_s3_png)
    print(f"✅ Mapa subido exitosamente a: s3://{bucket_name}/{ruta_s3_png}")

    # Subir el archivo CSV (lo guardamos dentro de una carpeta 'datos/')
    ruta_s3_csv = f'datos/{nombre_datos_csv}'
    s3.upload_file(nombre_datos_csv, bucket_name, ruta_s3_csv)
    print(f"✅ Datos CSV subidos exitosamente a: s3://{bucket_name}/{ruta_s3_csv}")

    print("\n🚀 ¡Proceso finalizado! Todos los archivos de tu maestría están respaldados en la nube.")

except FileNotFoundError as e:
    print(f"\n❌ Error: No se encontró el archivo local '{nombre_mapa_png}'. Asegúrate de haber ejecutado la celda que crea el map
except Exception as e:
    print(f"\n❌ Error al intentar subir a S3: {e}")
    print("💡 Tip: Verifica que el rol de IAM asociado a esta instancia de SageMaker tenga el permiso 's3:PutObject' para el bucke
```

PREDICCIONES CALI Y MEDELLIN MODELO 90/10

In [103…

```
from pycaret.regression import load_model
import awswrangler as wr

# 1. Cargar el modelo guardado de Bogotá
modelo = load_model('modelo_ivc_bogota_final_tuned')

# Obtener las columnas exactas de entrenamiento desde los pasos internos del pipeline de PyCaret
try:
    columnas_modelo = set(modelo.feature_names_in_)
except AttributeError:
    # Si es un pipeline completo de PyCaret, extraemos los nombres de entrada del estimador final o transformadores
    columnas_modelo = set(modelo[:-1].get_feature_names_out() if hasattr(modelo[:-1], 'get_feature_names_out') else modelo.n_featu

print("Modelo cargado correctamente. Verificando compatibilidad con nuevas ciudades...")

# 2. Cargar los datos de Medellín para comparar
```

```
bucket_entrada = 'mapastiposhapemanzanascalimedellin'
df_medellin = wr.s3.read_csv(f's3://{bucket_entrada}/CLEAN DATA PARA MODELAR/MEDELLIN CALI/df_medellin_ivc_filtrado.csv')
columnas_medellin = set(df_medellin.columns)

print(f"Columnas que tiene el CSV de Medellín: {len(columnas_medellin)}")
print(f"Columnas que el modelo de Bogotá necesita: {len(df_medellin.columns)}")
```

```
Transformation Pipeline and Model Successfully Loaded
Modelo cargado correctamente. Verificando compatibilidad con nuevas ciudades...
Columnas que tiene el CSV de Medellín: 21
Columnas que el modelo de Bogotá necesita: 21
```

In [113…

```
import os
import zipfile
import glob
import boto3
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point
import awswrangler as wr
from pycaret.regression import load_model, predict_model
import matplotlib.pyplot as plt
import contextily as ctx
from matplotlib_scalebar.scalebar import ScaleBar

# --- 1. CONFIGURACIÓN INICIAL ---
bucket_entrada = 'mapastiposhapemanzanascalimedellin'
bucket_salida = 'prediccionesbogota'

# --- 2. CARGA DEL MODELO Y DATOS TABULARES ---
print("Descargando modelo entrenado...")
wr.s3.download(path='s3://databogota/modelos/modelo_ivc_bogota_final_tuned.pkl', local_file='modelo_ivc_bogota_final_tuned.pkl')
modelo_produccion = load_model('modelo_ivc_bogota_final_tuned')

print("Cargando datos CSV de Medellín y Cali...")
df_medellin = wr.s3.read_csv(f's3://{bucket_entrada}/CLEAN DATA PARA MODELAR/MEDELLIN CALI/df_medellin_ivc_filtrado.csv')
df_cali = wr.s3.read_csv(f's3://{bucket_entrada}/CLEAN DATA PARA MODELAR/MEDELLIN CALI/df_cali_ivc_filtrado.csv')

# --- 3. ALINEACIÓN INTELIGENTE DE TIPOS DE DATOS ---
def alinear_con_modelo(df_nuevo, modelo):
    df_resultado = df_nuevo.copy()
    try:
        columnas_esperadas = modelo.steps[0][1].feature_names_in_
    except Exception:
        pipeline_obj = modelo.named_steps.get('actual_estimator', modelo)
        columnas_esperadas = getattr(pipeline_obj, 'feature_names_in_', df_nuevo.columns)

    for col in columnas_esperadas:
```

```
        if col not in df_resultado.columns:
            if col in ['NMB_LC_CM', 'TP_LC_CM']:
                df_resultado[col] = 'Desconocido'
            else:
                df_resultado[col] = 0.0

    return df_resultado[[c for c in columnas_esperadas if c in df_resultado.columns]]

df_medellin_listo = alinear_con_modelo(df_medellin, modelo_produccion)
df_cali_listo = alinear_con_modelo(df_cali, modelo_produccion)

for df_orig, df_dest in zip([df_medellin, df_cali], [df_medellin_listo, df_cali_listo]):
    if 'LATITUD' in df_orig.columns:
        df_dest['LATITUD'] = df_orig['LATITUD']
    if 'LONGITUD' in df_orig.columns:
        df_dest['LONGITUD'] = df_orig['LONGITUD']

# --- 4. INFERENCIA DEL MODELO ---
print("Ejecutando predicciones para Medellín...")
pred_medellin = predict_model(modelo_produccion, data=df_medellin_listo)
pred_medellin.rename(columns={'prediction_label': 'IVC_predicho'}, inplace=True)

print("Ejecutando predicciones para Cali...")
pred_cali = predict_model(modelo_produccion, data=df_cali_listo)
pred_cali.rename(columns={'prediction_label': 'IVC_predicho'}, inplace=True)

# --- 5. UBICACIÓN ESPACIAL (PUNTOS) ---
print("Mapeando coordenadas espaciales...")
gdf_pred_med = gpd.GeoDataFrame(
    pred_medellin,
    geometry=[Point(xy) for xy in zip(pred_medellin['LONGITUD'], pred_medellin['LATITUD'])],
    crs='EPSG:4326'
)
gdf_pred_cali = gpd.GeoDataFrame(
    pred_cali,
    geometry=[Point(xy) for xy in zip(pred_cali['LONGITUD'], pred_cali['LATITUD'])],
    crs='EPSG:4326'
)

# --- 6. EXTRACCIÓN DE SHAPEFILES Y GEODATABASES ---
print("Procesando geometrías base de Medellín...")
wr.s3.download(f's3://{bucket_entrada}/shp_lote_del_predio.zip', local_file='medellin.zip')
if os.path.exists('shp_medellin'):
    import shutil
    shutil.rmtree('shp_medellin')
os.makedirs('shp_medellin', exist_ok=True)
with zipfile.ZipFile('medellin.zip', 'r') as z:
    z.extractall('shp_medellin')
```

```
rutas_shp_med = glob.glob('shp_medellin/**/*.shp', recursive=True)
if rutas_shp_med:
    gdf_medellin_shp = gpd.read_file(rutas_shp_med[0])
    print(f"✅ Shapefile de Medellín cargado correctamente.")
else:
    raise FileNotFoundError("❌ No se encontró ningún archivo .shp en Medellín.")

print("Procesando geodatabase base de Cali (.rar)...")
wr.s3.download(f's3://{bucket_entrada}/GEODATABASE CATASTRAL CON CORTE A ENERO 2024.rar', local_file='cali.rar')
if os.path.exists('shp_cali'):
    import shutil
    shutil.rmtree('shp_cali')
os.makedirs('shp_cali', exist_ok=True)

# Descompresión mediante el binario unrar instalado en el entorno
os.system('unrar x -y cali.rar shp_cali/')

rutas_gdb = glob.glob('shp_cali/**/*.gdb', recursive=True)
rutas_shp = glob.glob('shp_cali/**/*.shp', recursive=True)

if rutas_gdb:
    gdf_cali_shp = gpd.read_file(rutas_gdb[0], driver='OpenFileGDB')
    print(f"✅ Geodatabase de Cali cargada exitosamente: {rutas_gdb[0]}")
elif rutas_shp:
    gdf_cali_shp = gpd.read_file(rutas_shp[0])
    print(f"✅ Shapefile de Cali cargado exitosamente: {rutas_shp[0]}")
else:
    raise FileNotFoundError("❌ No se encontró ningún archivo .gdb o .shp tras descomprimir el .rar de Cali.")

# Validación y asignación de CRS para Cali si viene indefinido
if gdf_cali_shp.crs is None:
    if gdf_cali_shp.total_bounds[2] > 180:
        gdf_cali_shp.set_crs('EPSG:3115', inplace=True) # Magna-Sirgas / Colombia West (metros)
    else:
        gdf_cali_shp.set_crs('EPSG:4326', inplace=True)

# --- 7. CRUCE ESPACIAL POLIGONAL (SJOIN + DISSOLVE) ---
print("Realizando cruce espacial poligonal...")
COL_ID_MEDELLIN = gdf_medellin_shp.columns[0]
COL_ID_CALI = gdf_cali_shp.columns[0]

gdf_pred_med = gdf_pred_med.to_crs(gdf_medellin_shp.crs)
gdf_pred_cali = gdf_pred_cali.to_crs(gdf_cali_shp.crs)

gdf_final_med = gpd.sjoin(gdf_pred_med, gdf_medellin_shp[[COL_ID_MEDELLIN, 'geometry']], how='left', predicate='intersects')
gdf_final_cali = gpd.sjoin(gdf_pred_cali, gdf_cali_shp[[COL_ID_CALI, 'geometry']], how='left', predicate='intersects')
```

```python
print(f"-> Puntos mapeados exitosamente en Medellín: {gdf_final_med[COL_ID_MEDELLIN].notna().sum():,}")
print(f"-> Puntos mapeados exitosamente en Cali: {gdf_final_cali[COL_ID_CALI].notna().sum():,}")

gdf_final_med = gdf_final_med[~gdf_final_med.index.duplicated(keep='first')]
gdf_final_cali = gdf_final_cali[~gdf_final_cali.index.duplicated(keep='first')]

gdf_mapa_medellin = gdf_final_med[[COL_ID_MEDELLIN, 'IVC_predicho', 'geometry']].dissolve(by=COL_ID_MEDELLIN, aggfunc={'IVC_predic
gdf_mapa_cali = gdf_final_cali[[COL_ID_CALI, 'IVC_predicho', 'geometry']].dissolve(by=COL_ID_CALI, aggfunc={'IVC_predicho': 'mean'

gdf_mapa_medellin = gdf_mapa_medellin[gdf_mapa_medellin['IVC_predicho'].notna()].copy()
gdf_mapa_cali = gdf_mapa_cali[gdf_mapa_cali['IVC_predicho'].notna()].copy()

# --- 8. CARTOGRAFÍA ACADÉMICA Y RESPALDO EN S3 ---
def mapear_y_guardar(gdf_poligonos, nombre_ciudad, archivo_png, archivo_csv):
    if len(gdf_poligonos) == 0:
        print(f"❌ Error: El GeoDataFrame para {nombre_ciudad} está vacío. No se puede generar el mapa.")
        return

    print(f"Generando mapa cartográfico para {nombre_ciudad}...")
    gdf_plot = gdf_poligonos.to_crs(epsg=3857)

    minx, miny, maxx, maxy = gdf_plot.total_bounds
    buffer_x = (maxx - minx) * 0.12
    buffer_y = (maxy - miny) * 0.12
    minx, maxx = minx - buffer_x, maxx + buffer_x
    miny, maxy = miny - buffer_y, maxy + buffer_y

    aspecto_mapa = (maxy - miny) / (maxx - minx)
    ancho_figura = 12
    alto_figura = ancho_figura * aspecto_mapa

    fig, ax = plt.subplots(figsize=(ancho_figura, alto_figura), dpi=300)
    fondo_color = '#F8F9FA'
    fig.patch.set_facecolor(fondo_color)
    ax.set_facecolor(fondo_color)

    gdf_plot.plot(column='IVC_predicho', cmap='YlOrRd', linewidth=0, edgecolor='none', alpha=0.9, ax=ax, legend=False)
    ctx.add_basemap(ax, source=ctx.providers.CartoDB.Positron, alpha=0.8)

    ax.set_xlim(minx, maxx)
    ax.set_ylim(miny, maxy)
    ax.set_aspect('equal')
    ax.set_axis_off()
    plt.subplots_adjust(top=0.90, bottom=0.12, left=0.02, right=0.82)

    fig.text(0.02, 0.95, "Índice de Vulnerabilidad Climática (IVC) Predicho", fontsize=20, fontweight='bold', color='#1A1A1A')
    fig.text(0.02, 0.92, f"{nombre_ciudad} — Transferencia Espacial del Modelo", fontsize=14, color='#4D4D4D')
```

```
    vmin, vmax = gdf_plot['IVC_predicho'].min(), gdf_plot['IVC_predicho'].max()
    sm = plt.cm.ScalarMappable(cmap='YlOrRd', norm=plt.Normalize(vmin=vmin, vmax=vmax))
    sm._A = []
    cax = fig.add_axes([0.85, 0.30, 0.025, 0.40])
    cb = fig.colorbar(sm, cax=cax)
    cb.set_label("Nivel de Vulnerabilidad (IVC)", fontsize=12, labelpad=12, fontweight='bold', color='#1A1A1A')
    cb.outline.set_visible(False)
    cb.ax.tick_params(size=0, labelsize=11, colors='#1A1A1A')

    ax.annotate('N', xy=(0.95, 0.96), xytext=(0.95, 0.89), xycoords='axes fraction', arrowprops=dict(facecolor='#1A1A1A', edgecolo
    escala = ScaleBar(dx=1, units="m", location='lower right', box_alpha=0.7, box_color='white', color='#1A1A1A', font_properties=
    ax.add_artist(escala)

    texto_metadatos = (
        "NOTAS METODOLÓGICAS:\n"
        f"• Algoritmo Predictivo: LightGBM (Transferencia de Dominio). Universo analizado: {gdf_plot.shape[0]:,} polígonos catastr
        "• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR). Proyección: Web Mercator (EPSG:3
    )
    fig.text(0.02, 0.02, texto_metadatos, fontsize=10, color='#4D4D4D', ha='left', linespacing=1.6)

    plt.savefig(archivo_png, dpi=300, bbox_inches='tight', pad_inches=0.1, facecolor=fondo_color)
    plt.show()

    gdf_poligonos.drop(columns=['geometry']).to_csv(archivo_csv, index=False)
    wr.s3.upload(archivo_png, f"s3://{bucket_salida}/mapas/{archivo_png}")
    wr.s3.upload(archivo_csv, f"s3://{bucket_salida}/datos/{archivo_csv}")
    print(f"✅ Resultados de {nombre_ciudad} respaldados exitosamente en S3.")

# Ejecución para Medellín y Cali
mapear_y_guardar(gdf_mapa_medellin, "Medellín, Antioquia", "mapa_ivc_medellin.png", "datos_ivc_medellin.csv")
mapear_y_guardar(gdf_mapa_cali, "Cali, Valle del Cauca", "mapa_ivc_cali.png", "datos_ivc_cali.csv")
```

```
Descargando modelo entrenado...
Transformation Pipeline and Model Successfully Loaded
Cargando datos CSV de Medellín y Cali...
Ejecutando predicciones para Medellín...
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	Light Gradient Boosting Machine	0.2264	0.0588	0.2425	0.0000	0.2135	nan

```
Ejecutando predicciones para Cali...
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	Light Gradient Boosting Machine	0.2203	0.0550	0.2345	0.0000	0.2074	nan

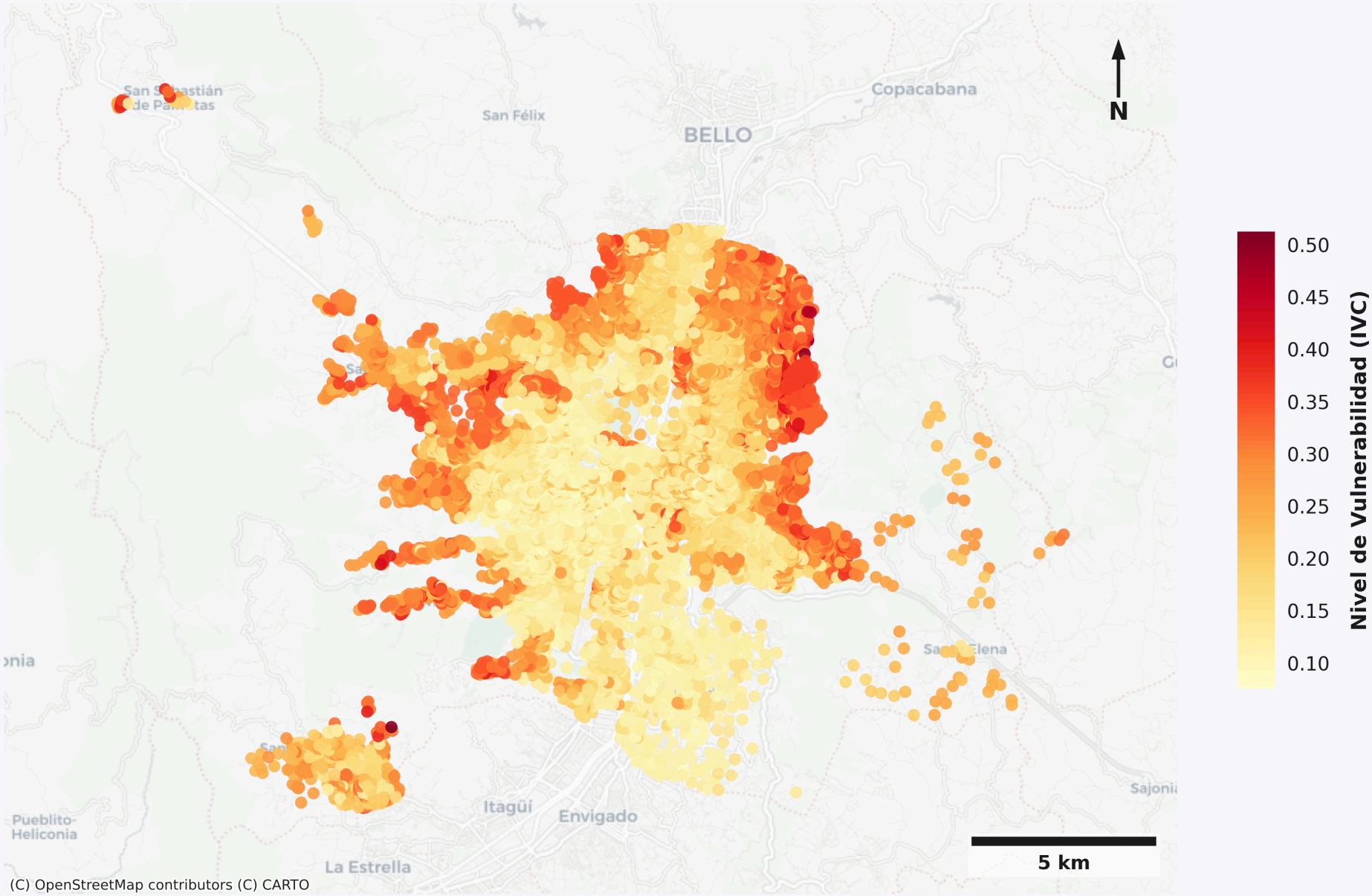
```
Mapeando coordenadas espaciales...
Procesando geometrías base de Medellín...
✅ Shapefile de Medellín cargado correctamente.
Procesando geodatabase base de Cali (.rar)...

UNRAR 7.23 freeware      Copyright (c) 1993-2026 Alexander Roshal

Extracting from cali.rar

Creating    shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024     OK
Creating    shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.TablesByName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000002.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000002.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000003.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000003.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000003.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.CatItemsByPhysicalName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.CatItemsByType.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.FDO_UUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.CatItemTypesByName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.CatItemTypesByParentTypeID.atx
OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.CatItemTypesByUUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.CatRelsByDestinationID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.CatRelsByOriginID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.CatRelsByType.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.FDO_UUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByBackwardLabel.atx
OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByDestItemTypeID.atx
OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByForwardLabel.atx  O
```

```
K
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByOriginItemTypeID.at
x  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByUUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.CONEXION.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.CONEXION.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.gdbtable  OK
```

```
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/gdb  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/timestamps  OK
All OK
✅ Geodatabase de Cali cargada exitosamente: shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb
Realizando cruce espacial poligonal...
-> Puntos mapeados exitosamente en Medellín: 14,083
-> Puntos mapeados exitosamente en Cali: 1,060
Generando mapa cartográfico para Medellín, Antioquia...
```

Índice de Vulnerabilidad Climática (IVC) Predicho

Medellín, Antioquia — Transferencia Espacial del Modelo



NOTAS METODOLÓGICAS:

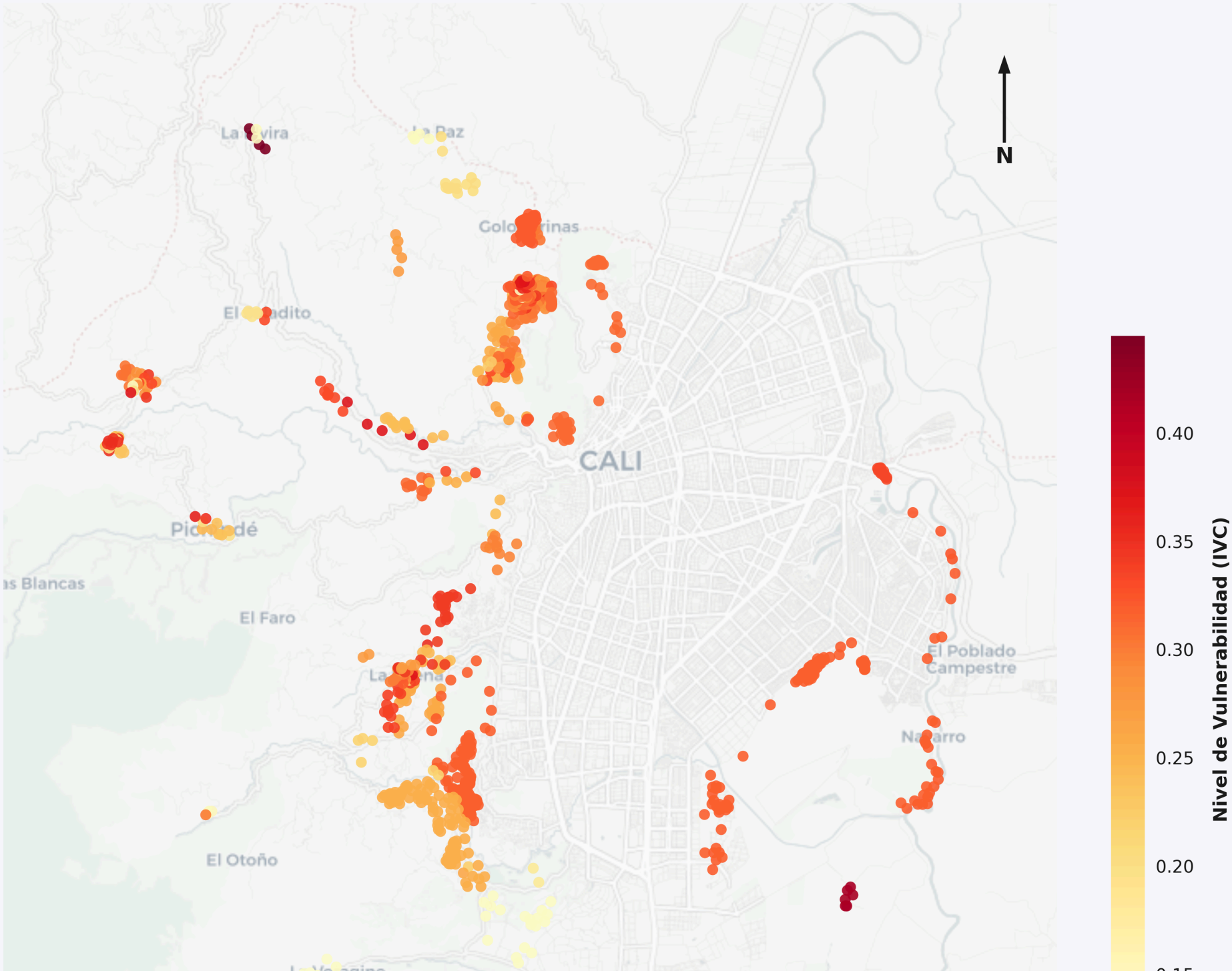
• Algoritmo Predictivo: LightGBM (Transferencia de Dominio). Universo analizado: 9,906 polígonos catastrales.

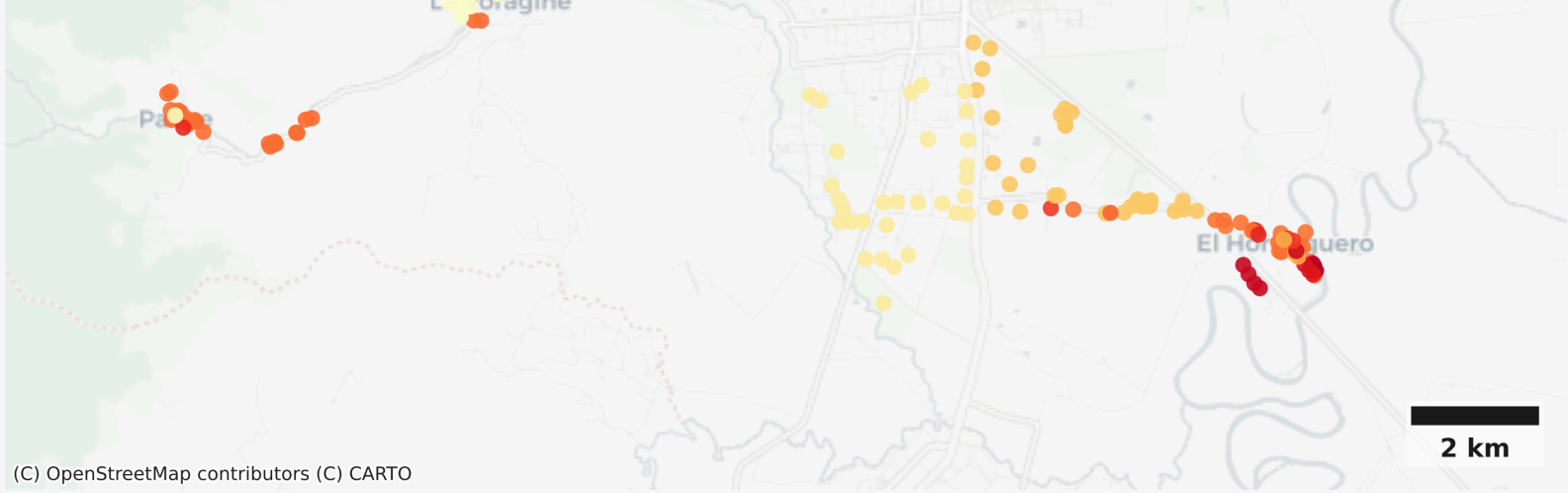
• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR). Proyección: Web Mercator (EPSG:3857).

✅ Resultados de Medellín, Antioquia respaldados exitosamente en S3.
Generando mapa cartográfico para Cali, Valle del Cauca...

Índice de Vulnerabilidad Climática (IVC) Predicho

Cali, Valle del Cauca — Transferencia Espacial del Modelo





NOTAS METODOLÓGICAS:
• Algoritmo Predictivo: LightGBM (Transferencia de Dominio). Universo analizado: 147 polígonos catastrales.
• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR). Proyección: Web Mercator (EPSG:3857).

✅ Resultados de Cali, Valle del Cauca respaldados exitosamente en S3.

In [114…

```python
import os
import zipfile
import glob
import boto3
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point
import awswrangler as wr
from pycaret.regression import load_model, predict_model
import matplotlib.pyplot as plt
import contextily as ctx
from matplotlib_scalebar.scalebar import ScaleBar

# --- 1. CONFIGURACIÓN INICIAL ---
bucket_entrada = 'mapastiposhapemanzanascalimedellin'
bucket_salida = 'prediccionesbogota'

# --- 2. CARGA DEL MODELO Y DATOS TABULARES ---
print("Descargando modelo entrenado...")
wr.s3.download(path='s3://databogota/modelos/modelo_ivc_bogota_final_tuned.pkl', local_file='modelo_ivc_bogota_final_tuned.pkl')
modelo_produccion = load_model('modelo_ivc_bogota_final_tuned')

print("Cargando datos CSV de Medellín y Cali...")
```

```python
df_medellin = wr.s3.read_csv(f's3://{bucket_entrada}/CLEAN DATA PARA MODELAR/MEDELLIN CALI/df_medellin_ivc_filtrado.csv')
df_cali = wr.s3.read_csv(f's3://{bucket_entrada}/CLEAN DATA PARA MODELAR/MEDELLIN CALI/df_cali_ivc_filtrado.csv')

# --- 3. ALINEACIÓN INTELIGENTE DE TIPOS DE DATOS ---
def alinear_con_modelo(df_nuevo, modelo):
    df_resultado = df_nuevo.copy()
    try:
        columnas_esperadas = modelo.steps[0][1].feature_names_in_
    except Exception:
        pipeline_obj = modelo.named_steps.get('actual_estimator', modelo)
        columnas_esperadas = getattr(pipeline_obj, 'feature_names_in_', df_nuevo.columns)

    for col in columnas_esperadas:
        if col not in df_resultado.columns:
            if col in ['NMB_LC_CM', 'TP_LC_CM']:
                df_resultado[col] = 'Desconocido'
            else:
                df_resultado[col] = 0.0

    return df_resultado[[c for c in columnas_esperadas if c in df_resultado.columns]]

df_medellin_listo = alinear_con_modelo(df_medellin, modelo_produccion)
df_cali_listo = alinear_con_modelo(df_cali, modelo_produccion)

for df_orig, df_dest in zip([df_medellin, df_cali], [df_medellin_listo, df_cali_listo]):
    if 'LATITUD' in df_orig.columns:
        df_dest['LATITUD'] = df_orig['LATITUD']
    if 'LONGITUD' in df_orig.columns:
        df_dest['LONGITUD'] = df_orig['LONGITUD']

# --- 4. INFERENCIA DEL MODELO ---
print("Ejecutando predicciones para Medellín...")
pred_medellin = predict_model(modelo_produccion, data=df_medellin_listo)
pred_medellin.rename(columns={'prediction_label': 'IVC_predicho'}, inplace=True)

print("Ejecutando predicciones para Cali...")
pred_cali = predict_model(modelo_produccion, data=df_cali_listo)
pred_cali.rename(columns={'prediction_label': 'IVC_predicho'}, inplace=True)

# --- 5. UBICACIÓN ESPACIAL (PUNTOS) ---
print("Mapeando coordenadas espaciales...")
gdf_pred_med = gpd.GeoDataFrame(
    pred_medellin,
    geometry=[Point(xy) for xy in zip(pred_medellin['LONGITUD'], pred_medellin['LATITUD'])],
    crs='EPSG:4326'
)
gdf_pred_cali = gpd.GeoDataFrame(
    pred_cali,
```

```
    geometry=[Point(xy) for xy in zip(pred_cali['LONGITUD'], pred_cali['LATITUD'])],
    crs='EPSG:4326'
)

# --- 6. EXTRACCIÓN DE SHAPEFILES Y GEODATABASES ---
print("Procesando geometrías base de Medellín...")
wr.s3.download(f's3://{bucket_entrada}/shp_lote_del_predio.zip', local_file='medellin.zip')
if os.path.exists('shp_medellin'):
    import shutil
    shutil.rmtree('shp_medellin')
os.makedirs('shp_medellin', exist_ok=True)
with zipfile.ZipFile('medellin.zip', 'r') as z:
    z.extractall('shp_medellin')

rutas_shp_med = glob.glob('shp_medellin/**/*.shp', recursive=True)
if rutas_shp_med:
    gdf_medellin_shp = gpd.read_file(rutas_shp_med[0])
    print(f"✅ Shapefile de Medellín cargado correctamente.")
else:
    raise FileNotFoundError("❌ No se encontró ningún archivo .shp en Medellín.")

print("Procesando geodatabase base de Cali (.rar)...")
wr.s3.download(f's3://{bucket_entrada}/GEODATABASE CATASTRAL CON CORTE A ENERO 2024.rar', local_file='cali.rar')
if os.path.exists('shp_cali'):
    import shutil
    shutil.rmtree('shp_cali')
os.makedirs('shp_cali', exist_ok=True)

os.system('unrar x -y cali.rar shp_cali/')

rutas_gdb = glob.glob('shp_cali/**/*.gdb', recursive=True)
rutas_shp = glob.glob('shp_cali/**/*.shp', recursive=True)

if rutas_gdb:
    gdf_cali_shp = gpd.read_file(rutas_gdb[0], driver='OpenFileGDB')
    print(f"✅ Geodatabase de Cali cargada exitosamente: {rutas_gdb[0]}")
elif rutas_shp:
    gdf_cali_shp = gpd.read_file(rutas_shp[0])
    print(f"✅ Shapefile de Cali cargado exitosamente: {rutas_shp[0]}")
else:
    raise FileNotFoundError("❌ No se encontró ningún archivo .gdb o .shp tras descomprimir el .rar de Cali.")

if gdf_cali_shp.crs is None:
    if gdf_cali_shp.total_bounds[2] > 180:
        gdf_cali_shp.set_crs('EPSG:3115', inplace=True)
    else:
        gdf_cali_shp.set_crs('EPSG:4326', inplace=True)
```

```python
# --- 7. CRUCE ESPACIAL AVANZADO (MÁXIMA COBERTURA) ---
print("Realizando cruce espacial optimizado...")
COL_ID_MEDELLIN = gdf_medellin_shp.columns[0]
COL_ID_CALI = gdf_cali_shp.columns[0]

# Medellín mantiene sjoin estándar por intersección
gdf_pred_med = gdf_pred_med.to_crs(gdf_medellin_shp.crs)
gdf_final_med = gpd.sjoin(gdf_pred_med, gdf_medellin_shp[[COL_ID_MEDELLIN, 'geometry']], how='left', predicate='intersects')

# CALI: Se proyecta a metros (EPSG:3115 - Magna-Sirgas Cali/Occidente) y se usa SJOIN_NEAREST con 35m de tolerancia máxima
CRS_METROS_CALI = 'EPSG:3115'
if gdf_cali_shp.crs.to_epsg() != 3115:
    gdf_cali_shp = gdf_cali_shp.to_crs(CRS_METROS_CALI)
gdf_pred_cali = gdf_pred_cali.to_crs(CRS_METROS_CALI)

print("Ejecutando sjoin_nearest para Cali (capturando puntos cercanos a manzanas)...")
gdf_final_cali = gpd.sjoin_nearest(
    gdf_pred_cali,
    gdf_cali_shp[[COL_ID_CALI, 'geometry']],
    how='left',
    max_distance=35.0, # Tolerancia de hasta 35 metros para rescatar puntos en calles o desfasados
    distance_col="distancia_buffer"
)

print(f"-> Puntos mapeados exitosamente en Medellín: {gdf_final_med[COL_ID_MEDELLIN].notna().sum():,}")
print(f"-> Puntos mapeados exitosamente en Cali (Optimizado): {gdf_final_cali[COL_ID_CALI].notna().sum():,}")

gdf_final_med = gdf_final_med[~gdf_final_med.index.duplicated(keep='first')]
gdf_final_cali = gdf_final_cali[~gdf_final_cali.index.duplicated(keep='first')]

gdf_mapa_medellin = gdf_final_med[[COL_ID_MEDELLIN, 'IVC_predicho', 'geometry']].dissolve(by=COL_ID_MEDELLIN, aggfunc={'IVC_predic
gdf_mapa_cali = gdf_final_cali[[COL_ID_CALI, 'IVC_predicho', 'geometry']].dissolve(by=COL_ID_CALI, aggfunc={'IVC_predicho': 'mean'

gdf_mapa_medellin = gdf_mapa_medellin[gdf_mapa_medellin['IVC_predicho'].notna()].copy()
gdf_mapa_cali = gdf_mapa_cali[gdf_mapa_cali['IVC_predicho'].notna()].copy()

# --- 8. CARTOGRAFÍA ACADÉMICA Y RESPALDO EN S3 ---
def mapear_y_guardar(gdf_poligonos, nombre_ciudad, archivo_png, archivo_csv):
    if len(gdf_poligonos) == 0:
        print(f"❌ Error: El GeoDataFrame para {nombre_ciudad} está vacío. No se puede generar el mapa.")
        return

    print(f"Generando mapa cartográfico para {nombre_ciudad}...")
    gdf_plot = gdf_poligonos.to_crs(epsg=3857)

    minx, miny, maxx, maxy = gdf_plot.total_bounds
    buffer_x = (maxx - minx) * 0.12
    buffer_y = (maxy - miny) * 0.12
```

```
minx, maxx = minx - buffer_x, maxx + buffer_x
miny, maxy = miny - buffer_y, maxy + buffer_y

aspecto_mapa = (maxy - miny) / (maxx - minx)
ancho_figura = 12
alto_figura = ancho_figura * aspecto_mapa

fig, ax = plt.subplots(figsize=(ancho_figura, alto_figura), dpi=300)
fondo_color = '#F8F9FA'
fig.patch.set_facecolor(fondo_color)
ax.set_facecolor(fondo_color)

gdf_plot.plot(column='IVC_predicho', cmap='YlOrRd', linewidth=0, edgecolor='none', alpha=0.9, ax=ax, legend=False)
ctx.add_basemap(ax, source=ctx.providers.CartoDB.Positron, alpha=0.8)

ax.set_xlim(minx, maxx)
ax.set_ylim(miny, maxy)
ax.set_aspect('equal')
ax.set_axis_off()
plt.subplots_adjust(top=0.90, bottom=0.12, left=0.02, right=0.82)

fig.text(0.02, 0.95, "Índice de Vulnerabilidad Climática (IVC) Predicho", fontsize=20, fontweight='bold', color='#1A1A1A')
fig.text(0.02, 0.92, f"{nombre_ciudad} — Transferencia Espacial del Modelo", fontsize=14, color='#4D4D4D')

vmin, vmax = gdf_plot['IVC_predicho'].min(), gdf_plot['IVC_predicho'].max()
sm = plt.cm.ScalarMappable(cmap='YlOrRd', norm=plt.Normalize(vmin=vmin, vmax=vmax))
sm._A = []
cax = fig.add_axes([0.85, 0.30, 0.025, 0.40])
cb = fig.colorbar(sm, cax=cax)
cb.set_label("Nivel de Vulnerabilidad (IVC)", fontsize=12, labelpad=12, fontweight='bold', color='#1A1A1A')
cb.outline.set_visible(False)
cb.ax.tick_params(size=0, labelsize=11, colors='#1A1A1A')

ax.annotate('N', xy=(0.95, 0.96), xytext=(0.95, 0.89), xycoords='axes fraction', arrowprops=dict(facecolor='#1A1A1A', edgecolo
escala = ScaleBar(dx=1, units="m", location='lower right', box_alpha=0.7, box_color='white', color='#1A1A1A', font_properties=
ax.add_artist(escala)

texto_metadatos = (
    "NOTAS METODOLÓGICAS:\n"
    f"• Algoritmo Predictivo: LightGBM (Transferencia de Dominio). Universo analizado: {gdf_plot.shape[0]:,} polígonos catastr
    "• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR). Proyección: Web Mercator (EPSG:3
)
fig.text(0.02, 0.02, texto_metadatos, fontsize=10, color='#4D4D4D', ha='left', linespacing=1.6)

plt.savefig(archivo_png, dpi=300, bbox_inches='tight', pad_inches=0.1, facecolor=fondo_color)
plt.show()

gdf_poligonos.drop(columns=['geometry']).to_csv(archivo_csv, index=False)
```

```
    wr.s3.upload(archivo_png, f"s3://{bucket_salida}/mapas/{archivo_png}")
    wr.s3.upload(archivo_csv, f"s3://{bucket_salida}/datos/{archivo_csv}")
    print(f"✅ Resultados de {nombre_ciudad} respaldados exitosamente en S3.")

# Ejecución para Medellín y Cali
mapear_y_guardar(gdf_mapa_medellin, "Medellín, Antioquia", "mapa_ivc_medellin.png", "datos_ivc_medellin.csv")
mapear_y_guardar(gdf_mapa_cali, "Cali, Valle del Cauca", "mapa_ivc_cali.png", "datos_ivc_cali.csv")
```

```
Descargando modelo entrenado...
Transformation Pipeline and Model Successfully Loaded
Cargando datos CSV de Medellín y Cali...
Ejecutando predicciones para Medellín...
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	Light Gradient Boosting Machine	0.2264	0.0588	0.2425	0.0000	0.2135	nan

```
Ejecutando predicciones para Cali...
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	Light Gradient Boosting Machine	0.2203	0.0550	0.2345	0.0000	0.2074	nan

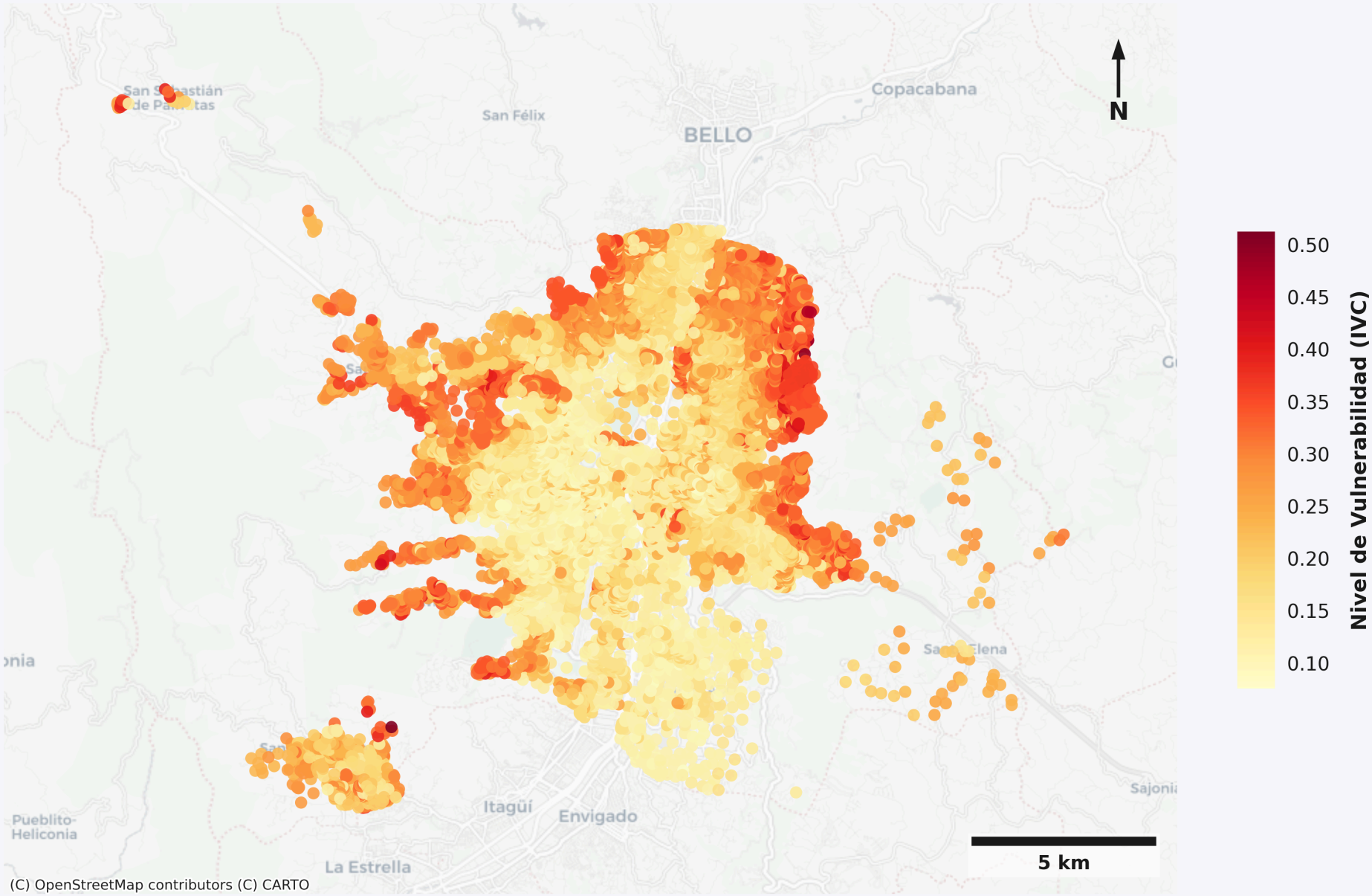
```
Mapeando coordenadas espaciales...
Procesando geometrías base de Medellín...
✅ Shapefile de Medellín cargado correctamente.
Procesando geodatabase base de Cali (.rar)...

UNRAR 7.23 freeware      Copyright (c) 1993-2026 Alexander Roshal

Extracting from cali.rar

Creating    shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024     OK
Creating    shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.TablesByName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000002.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000002.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000003.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000003.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000003.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.CatItemsByPhysicalName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.CatItemsByType.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.FDO_UUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.CatItemTypesByName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.CatItemTypesByParentTypeID.atx
OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.CatItemTypesByUUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.CatRelsByDestinationID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.CatRelsByOriginID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.CatRelsByType.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.FDO_UUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByBackwardLabel.atx
OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByDestItemTypeID.atx
OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByForwardLabel.atx  O
```

```
K
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByOriginItemTypeID.at
x  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByUUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.CONEXION.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.CONEXION.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.gdbtable  OK
```

```
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/gdb  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/timestamps  OK
All OK
✅ Geodatabase de Cali cargada exitosamente: shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb
Realizando cruce espacial optimizado...
Ejecutando sjoin_nearest para Cali (capturando puntos cercanos a manzanas)...
-> Puntos mapeados exitosamente en Medellín: 14,083
-> Puntos mapeados exitosamente en Cali (Optimizado): 1,213
Generando mapa cartográfico para Medellín, Antioquia...
```

Índice de Vulnerabilidad Climática (IVC) Predicho

Medellín, Antioquia — Transferencia Espacial del Modelo



NOTAS METODOLÓGICAS:

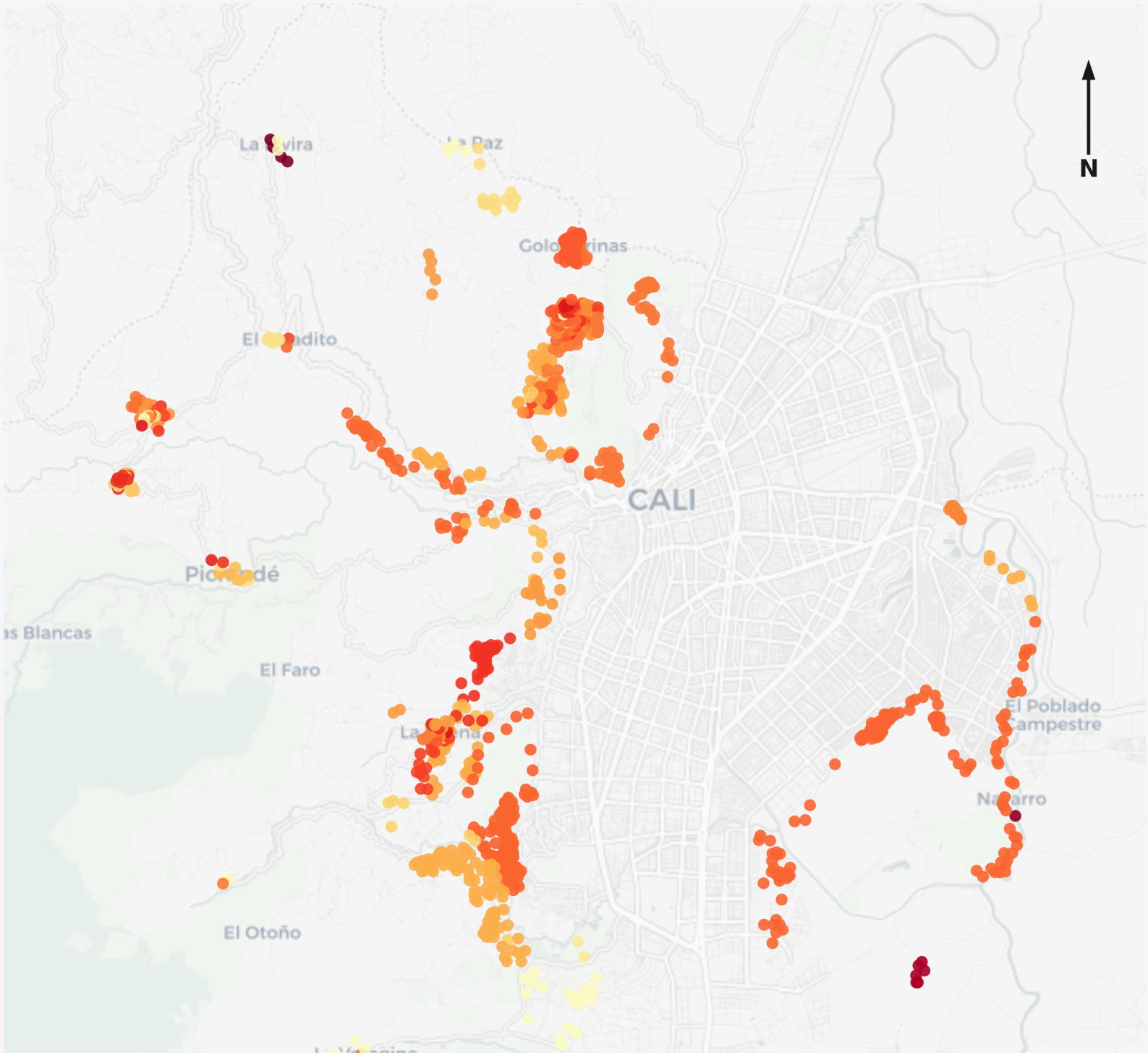
• Algoritmo Predictivo: LightGBM (Transferencia de Dominio). Universo analizado: 9,906 polígonos catastrales.

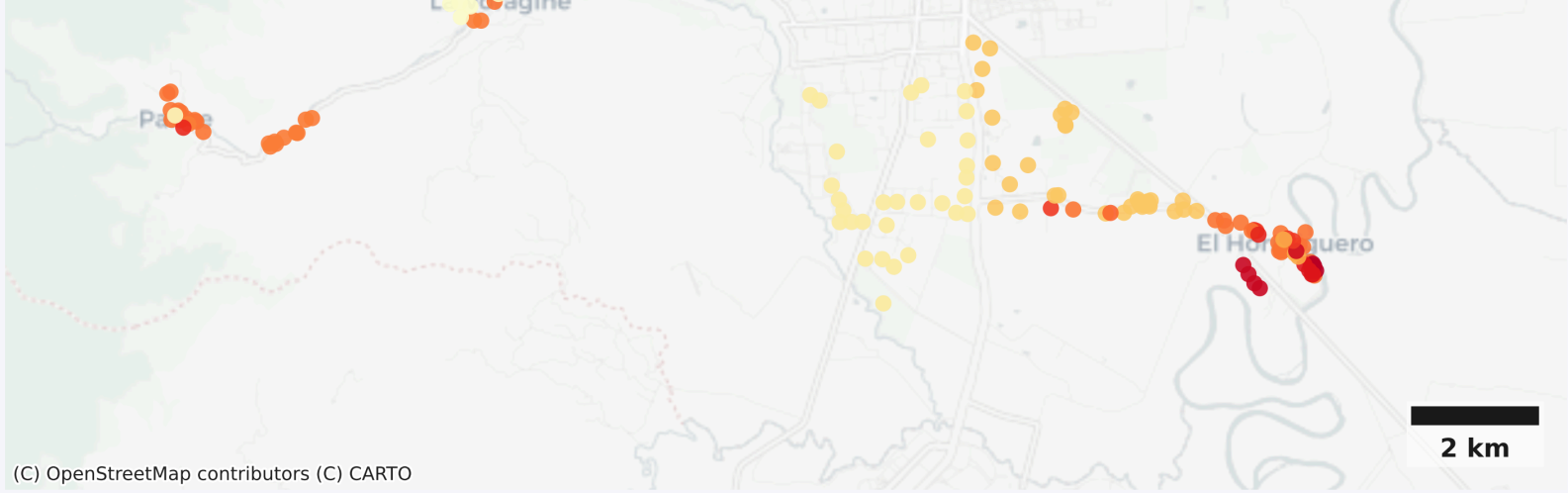
• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR). Proyección: Web Mercator (EPSG:3857).

✅ Resultados de Medellín, Antioquia respaldados exitosamente en S3.
Generando mapa cartográfico para Cali, Valle del Cauca...

Índice de Vulnerabilidad Climática (IVC) Predicho

Cali, Valle del Cauca — Transferencia Espacial del Modelo





NOTAS METODOLÓGICAS:
• Algoritmo Predictivo: LightGBM (Transferencia de Dominio). Universo analizado: 148 polígonos catastrales.
• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR). Proyección: Web Mercator (EPSG:3857).

✅ Resultados de Cali, Valle del Cauca respaldados exitosamente en S3.

In [117…

```python
import os
import zipfile
import glob
import boto3
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point
import awswrangler as wr
from pycaret.regression import load_model, predict_model
import matplotlib.pyplot as plt
import contextily as ctx
from matplotlib_scalebar.scalebar import ScaleBar

# --- 1. CONFIGURACIÓN INICIAL ---
bucket_entrada = 'mapastiposhapemanzanascalimedellin'
bucket_salida = 'prediccionesbogota'

# --- 2. CARGA DEL MODELO Y DATOS TABULARES ---
print("Descargando modelo entrenado...")
wr.s3.download(path='s3://databogota/modelos/modelo_ivc_bogota_final_tuned.pkl', local_file='modelo_ivc_bogota_final_tuned.pkl')
modelo_produccion = load_model('modelo_ivc_bogota_final_tuned')

print("Cargando datos CSV de Medellín y Cali...")
```

```python
df_medellin = wr.s3.read_csv(f's3://{bucket_entrada}/CLEAN DATA PARA MODELAR/MEDELLIN CALI/df_medellin_ivc_filtrado.csv')
df_cali = wr.s3.read_csv(f's3://{bucket_entrada}/CLEAN DATA PARA MODELAR/MEDELLIN CALI/df_cali_ivc_filtrado.csv')

# --- 3. ALINEACIÓN INTELIGENTE DE TIPOS DE DATOS ---
def alinear_con_modelo(df_nuevo, modelo):
    df_resultado = df_nuevo.copy()
    try:
        columnas_esperadas = modelo.steps[0][1].feature_names_in_
    except Exception:
        pipeline_obj = modelo.named_steps.get('actual_estimator', modelo)
        columnas_esperadas = getattr(pipeline_obj, 'feature_names_in_', df_nuevo.columns)

    for col in columnas_esperadas:
        if col not in df_resultado.columns:
            if col in ['NMB_LC_CM', 'TP_LC_CM']:
                df_resultado[col] = 'Desconocido'
            else:
                df_resultado[col] = 0.0

    return df_resultado[[c for c in columnas_esperadas if c in df_resultado.columns]]

df_medellin_listo = alinear_con_modelo(df_medellin, modelo_produccion)
df_cali_listo = alinear_con_modelo(df_cali, modelo_produccion)

for df_orig, df_dest in zip([df_medellin, df_cali], [df_medellin_listo, df_cali_listo]):
    if 'LATITUD' in df_orig.columns:
        df_dest['LATITUD'] = df_orig['LATITUD']
    if 'LONGITUD' in df_orig.columns:
        df_dest['LONGITUD'] = df_orig['LONGITUD']

# --- 4. INFERENCIA DEL MODELO ---
print("Ejecutando predicciones para Medellín...")
pred_medellin = predict_model(modelo_produccion, data=df_medellin_listo)
pred_medellin.rename(columns={'prediction_label': 'IVC_predicho'}, inplace=True)

print("Ejecutando predicciones para Cali...")
pred_cali = predict_model(modelo_produccion, data=df_cali_listo)
pred_cali.rename(columns={'prediction_label': 'IVC_predicho'}, inplace=True)

# --- 5. UBICACIÓN ESPACIAL (PUNTOS) ---
print("Mapeando coordenadas espaciales...")
gdf_pred_med = gpd.GeoDataFrame(
    pred_medellin,
    geometry=[Point(xy) for xy in zip(pred_medellin['LONGITUD'], pred_medellin['LATITUD'])],
    crs='EPSG:4326'
)
gdf_pred_cali = gpd.GeoDataFrame(
    pred_cali,
```

```
    geometry=[Point(xy) for xy in zip(pred_cali['LONGITUD'], pred_cali['LATITUD'])],
    crs='EPSG:4326'
)

# --- 6. EXTRACCIÓN DE SHAPEFILES Y GEODATABASES ---
print("Procesando geometrías base de Medellín...")
wr.s3.download(f's3://{bucket_entrada}/shp_lote_del_predio.zip', local_file='medellin.zip')
if os.path.exists('shp_medellin'):
    import shutil
    shutil.rmtree('shp_medellin')
os.makedirs('shp_medellin', exist_ok=True)
with zipfile.ZipFile('medellin.zip', 'r') as z:
    z.extractall('shp_medellin')

rutas_shp_med = glob.glob('shp_medellin/**/*.shp', recursive=True)
if rutas_shp_med:
    gdf_medellin_shp = gpd.read_file(rutas_shp_med[0])
    print(f"✅ Shapefile de Medellín cargado correctamente.")
else:
    raise FileNotFoundError("❌ No se encontró ningún archivo .shp en Medellín.")

print("Procesando geodatabase base de Cali (.rar)...")
wr.s3.download(f's3://{bucket_entrada}/GEODATABASE CATASTRAL CON CORTE A ENERO 2024.rar', local_file='cali.rar')
if os.path.exists('shp_cali'):
    import shutil
    shutil.rmtree('shp_cali')
os.makedirs('shp_cali', exist_ok=True)

os.system('unrar x -y cali.rar shp_cali/')

rutas_gdb = glob.glob('shp_cali/**/*.gdb', recursive=True)
rutas_shp = glob.glob('shp_cali/**/*.shp', recursive=True)

if rutas_gdb:
    gdf_cali_shp = gpd.read_file(rutas_gdb[0], driver='OpenFileGDB')
    print(f"✅ Geodatabase de Cali cargada exitosamente: {rutas_gdb[0]}")
elif rutas_shp:
    gdf_cali_shp = gpd.read_file(rutas_shp[0])
    print(f"✅ Shapefile de Cali cargado exitosamente: {rutas_shp[0]}")
else:
    raise FileNotFoundError("❌ No se encontró ningún archivo .gdb o .shp tras descomprimir el .rar de Cali.")

if gdf_cali_shp.crs is None:
    if gdf_cali_shp.total_bounds[2] > 180:
        gdf_cali_shp.set_crs('EPSG:3115', inplace=True)
    else:
        gdf_cali_shp.set_crs('EPSG:4326', inplace=True)
```

```python
# --- 7. CRUCE ESPACIAL DE MÁXIMA COBERTURA (BUFFER MÉTRICO PARA CALI) ---
print("Realizando cruce espacial optimizado con máxima cobertura...")
COL_ID_MEDELLIN = gdf_medellin_shp.columns[0]
COL_ID_CALI = gdf_cali_shp.columns[0]

# Medellín mantiene sjoin estándar por intersección
gdf_pred_med = gdf_pred_med.to_crs(gdf_medellin_shp.crs)
gdf_final_med = gpd.sjoin(gdf_pred_med, gdf_medellin_shp[[COL_ID_MEDELLIN, 'geometry']], how='left', predicate='intersects')

# CALI: Proyección a metros (EPSG:3115 - Magna-Sirgas Cali/Occidente)
CRS_METROS_CALI = 'EPSG:3115'
if gdf_cali_shp.crs.to_epsg() != 3115:
    gdf_cali_shp = gdf_cali_shp.to_crs(CRS_METROS_CALI)
gdf_pred_cali = gdf_pred_cali.to_crs(CRS_METROS_CALI)

# TRUCO MÁXIMO: Ampliamos las manzanas de Cali 18 metros hacia afuera
# para capturar puntos en calles, andenes o con desfase de geocodificación.
print("Aplicando expansión geométrica (buffer de 18m) a las manzanas de Cali...")
gdf_cali_shp_buffered = gdf_cali_shp.copy()
gdf_cali_shp_buffered['geometry'] = gdf_cali_shp_buffered.geometry.buffer(18.0)

gdf_final_cali = gpd.sjoin(
    gdf_pred_cali,
    gdf_cali_shp_buffered[[COL_ID_CALI, 'geometry']],
    how='left',
    predicate='intersects'
)

print(f"-> Puntos mapeados exitosamente en Medellín: {gdf_final_med[COL_ID_MEDELLIN].notna().sum():,}")
print(f"-> Puntos mapeados exitosamente en Cali (Con Buffer Máximo): {gdf_final_cali[COL_ID_CALI].notna().sum():,}")

gdf_final_med = gdf_final_med[~gdf_final_med.index.duplicated(keep='first')]
gdf_final_cali = gdf_final_cali[~gdf_final_cali.index.duplicated(keep='first')]

gdf_mapa_medellin = gdf_final_med[[COL_ID_MEDELLIN, 'IVC_predicho', 'geometry']].dissolve(by=COL_ID_MEDELLIN, aggfunc={'IVC_predic

# Para el disolver de Cali, usamos las geometrías originales (sin buffer) para mantener la cartografía limpia y exacta
gdf_final_cali_original = gdf_final_cali.merge(gdf_cali_shp[[COL_ID_CALI, 'geometry']], on=COL_ID_CALI, suffixes=('_buff', ''))
# Reemplazamos la geometría por la original
gdf_final_cali_original['geometry'] = gdf_final_cali_original['geometry'].fillna(gdf_final_cali_original['geometry_buff'])

gdf_mapa_cali = gdf_final_cali_original[[COL_ID_CALI, 'IVC_predicho', 'geometry']].dissolve(by=COL_ID_CALI, aggfunc={'IVC_predicho

gdf_mapa_medellin = gdf_mapa_medellin[gdf_mapa_medellin['IVC_predicho'].notna()].copy()
gdf_mapa_cali = gdf_mapa_cali[gdf_mapa_cali['IVC_predicho'].notna()].copy()

# --- 8. CARTOGRAFÍA ACADÉMICA Y RESPALDO EN S3 ---
def mapear_y_guardar(gdf_poligonos, nombre_ciudad, archivo_png, archivo_csv):
```

```
if len(gdf_poligonos) == 0:
    print(f"❌ Error: El GeoDataFrame para {nombre_ciudad} está vacío. No se puede generar el mapa.")
    return

print(f"Generando mapa cartográfico para {nombre_ciudad}...")
gdf_plot = gdf_poligonos.to_crs(epsg=3857)

minx, miny, maxx, maxy = gdf_plot.total_bounds
buffer_x = (maxx - minx) * 0.12
buffer_y = (maxy - miny) * 0.12
minx, maxx = minx - buffer_x, maxx + buffer_x
miny, maxy = miny - buffer_y, maxy + buffer_y

aspecto_mapa = (maxy - miny) / (maxx - minx)
ancho_figura = 12
alto_figura = ancho_figura * aspecto_mapa

fig, ax = plt.subplots(figsize=(ancho_figura, alto_figura), dpi=300)
fondo_color = '#F8F9FA'
fig.patch.set_facecolor(fondo_color)
ax.set_facecolor(fondo_color)

gdf_plot.plot(column='IVC_predicho', cmap='YlOrRd', linewidth=0, edgecolor='none', alpha=0.9, ax=ax, legend=False)
ctx.add_basemap(ax, source=ctx.providers.CartoDB.Positron, alpha=0.8)

ax.set_xlim(minx, maxx)
ax.set_ylim(miny, maxy)
ax.set_aspect('equal')
ax.set_axis_off()
plt.subplots_adjust(top=0.90, bottom=0.12, left=0.02, right=0.82)

fig.text(0.02, 0.95, "Índice de Vulnerabilidad Climática (IVC) Predicho", fontsize=20, fontweight='bold', color='#1A1A1A')
fig.text(0.02, 0.92, f"{nombre_ciudad} — Transferencia Espacial del Modelo", fontsize=14, color='#4D4D4D')

vmin, vmax = gdf_plot['IVC_predicho'].min(), gdf_plot['IVC_predicho'].max()
sm = plt.cm.ScalarMappable(cmap='YlOrRd', norm=plt.Normalize(vmin=vmin, vmax=vmax))
sm._A = []
cax = fig.add_axes([0.85, 0.30, 0.025, 0.40])
cb = fig.colorbar(sm, cax=cax)
cb.set_label("Nivel de Vulnerabilidad (IVC)", fontsize=12, labelpad=12, fontweight='bold', color='#1A1A1A')
cb.outline.set_visible(False)
cb.ax.tick_params(size=0, labelsize=11, colors='#1A1A1A')

ax.annotate('N', xy=(0.95, 0.96), xytext=(0.95, 0.89), xycoords='axes fraction', arrowprops=dict(facecolor='#1A1A1A', edgecolo
escala = ScaleBar(dx=1, units="m", location='lower right', box_alpha=0.7, box_color='white', color='#1A1A1A', font_properties=
ax.add_artist(escala)

texto_metadatos = (
```

```
        "NOTAS METODOLÓGICAS:\n"
        f"• Algoritmo Predictivo: LightGBM (Transferencia de Dominio). Universo analizado: {gdf_plot.shape[0]:,} polígonos catastr
        "• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR). Proyección: Web Mercator (EPSG:3
    )
    fig.text(0.02, 0.02, texto_metadatos, fontsize=10, color='#4D4D4D', ha='left', linespacing=1.6)

    plt.savefig(archivo_png, dpi=300, bbox_inches='tight', pad_inches=0.1, facecolor=fondo_color)
    plt.show()

    gdf_poligonos.drop(columns=['geometry']).to_csv(archivo_csv, index=False)
    wr.s3.upload(archivo_png, f"s3://{bucket_salida}/mapas/{archivo_png}")
    wr.s3.upload(archivo_csv, f"s3://{bucket_salida}/datos/{archivo_csv}")
    print(f"✅ Resultados de {nombre_ciudad} respaldados exitosamente en S3.")

# Ejecución para Medellín y Cali
mapear_y_guardar(gdf_mapa_medellin, "Medellín, Antioquia", "mapa_ivc_medellin.png", "datos_ivc_medellin.csv")
mapear_y_guardar(gdf_mapa_cali, "Cali, Valle del Cauca", "mapa_ivc_cali.png", "datos_ivc_cali.csv")
```

```
Descargando modelo entrenado...
Transformation Pipeline and Model Successfully Loaded
Cargando datos CSV de Medellín y Cali...
Ejecutando predicciones para Medellín...
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	Light Gradient Boosting Machine	0.2264	0.0588	0.2425	0.0000	0.2135	nan

```
Ejecutando predicciones para Cali...
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	Light Gradient Boosting Machine	0.2203	0.0550	0.2345	0.0000	0.2074	nan

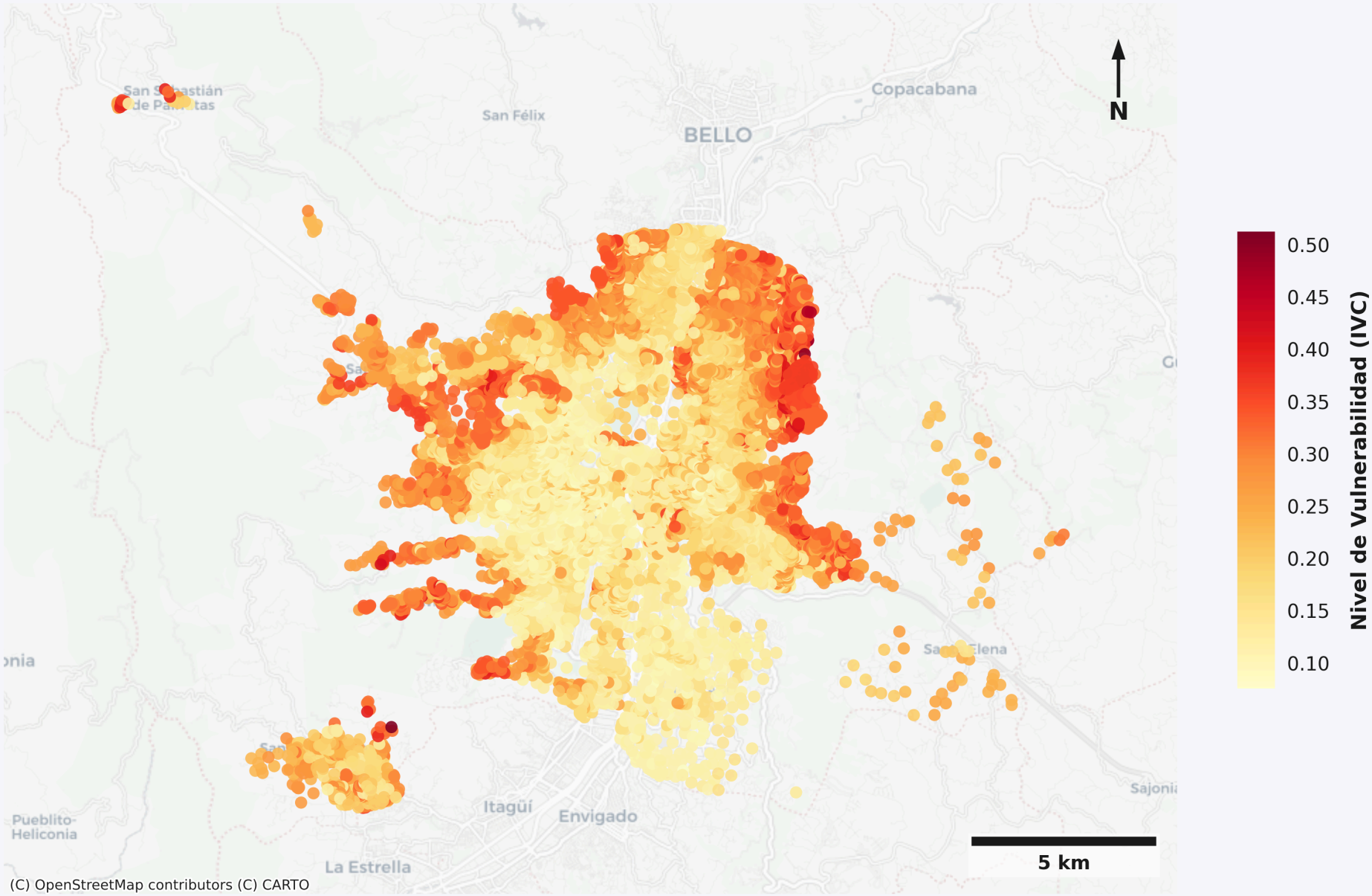
```
Mapeando coordenadas espaciales...
Procesando geometrías base de Medellín...
✅ Shapefile de Medellín cargado correctamente.
Procesando geodatabase base de Cali (.rar)...

UNRAR 7.23 freeware      Copyright (c) 1993-2026 Alexander Roshal

Extracting from cali.rar

Creating    shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024     OK
Creating    shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000001.TablesByName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000002.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000002.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000003.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000003.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000003.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.CatItemsByPhysicalName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.CatItemsByType.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.FDO_UUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000004.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.CatItemTypesByName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.CatItemTypesByParentTypeID.atx
OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.CatItemTypesByUUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000005.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.CatRelsByDestinationID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.CatRelsByOriginID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.CatRelsByType.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.FDO_UUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000006.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByBackwardLabel.atx
OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByDestItemTypeID.atx
OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByForwardLabel.atx  O
```

```
K
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByName.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByOriginItemTypeID.at
x  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.CatRelTypesByUUID.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000007.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000009.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.CONEXION.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000a.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000b.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000c.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000d.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000e.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a0000000f.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.CONEXION.atx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.gdbtable  OK
```

```
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000010.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000011.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.freelist  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.gdbindexes  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.gdbtable  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.gdbtablx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/a00000012.spx  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/gdb  OK
Extracting  shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb/timestamps  OK
All OK
✅ Geodatabase de Cali cargada exitosamente: shp_cali/GEODATABASE CATASTRAL CON CORTE A ENERO 2024/CATASTRAL_BDG_2024.gdb
Realizando cruce espacial optimizado con máxima cobertura...
Aplicando expansión geométrica (buffer de 18m) a las manzanas de Cali...
-> Puntos mapeados exitosamente en Medellín: 14,083
-> Puntos mapeados exitosamente en Cali (Con Buffer Máximo): 1,296
Generando mapa cartográfico para Medellín, Antioquia...
```

Índice de Vulnerabilidad Climática (IVC) Predicho

Medellín, Antioquia — Transferencia Espacial del Modelo



NOTAS METODOLÓGICAS:

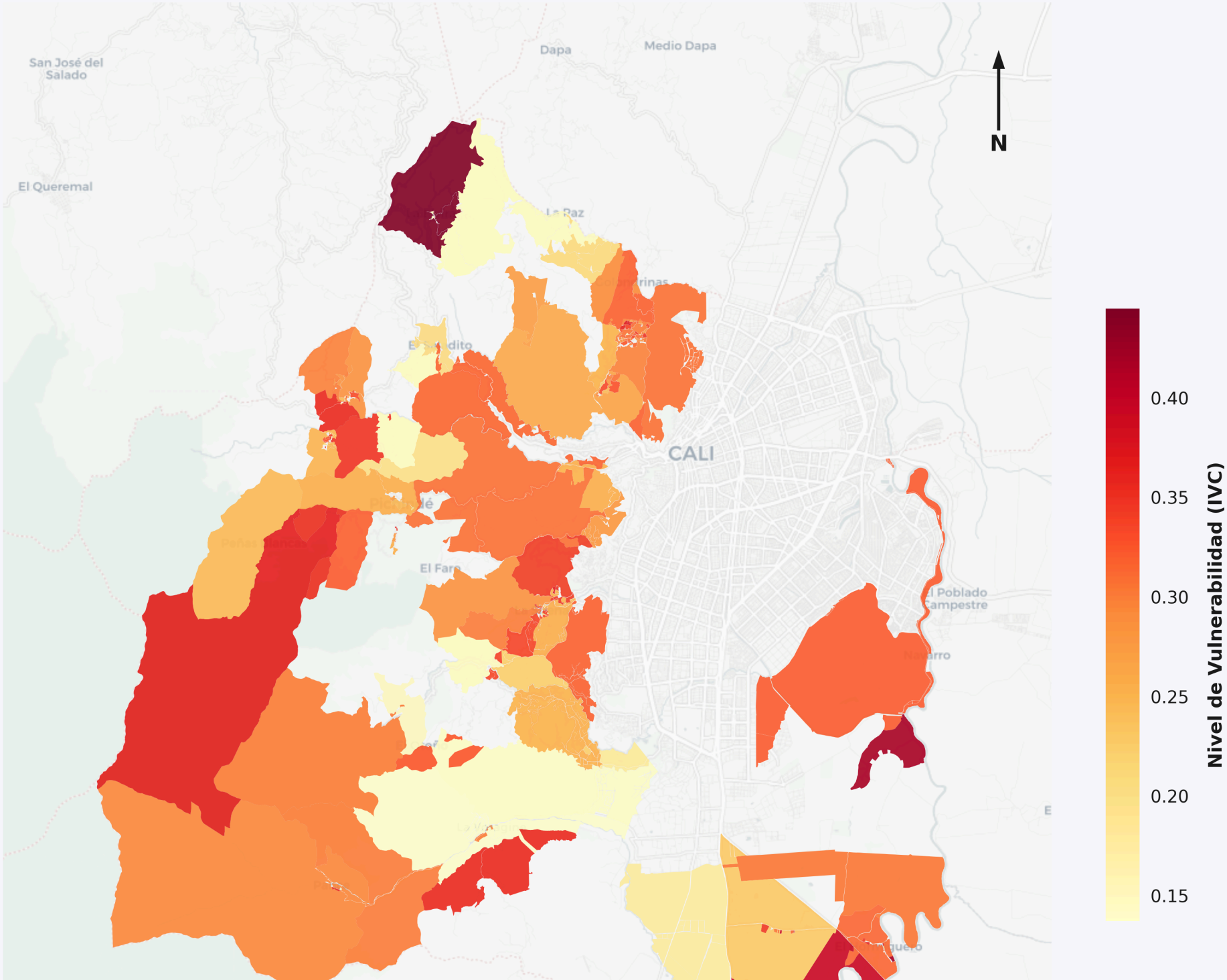
• Algoritmo Predictivo: LightGBM (Transferencia de Dominio). Universo analizado: 9,906 polígonos catastrales.

• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR). Proyección: Web Mercator (EPSG:3857).

✅ Resultados de Medellín, Antioquia respaldados exitosamente en S3.
Generando mapa cartográfico para Cali, Valle del Cauca...

Índice de Vulnerabilidad Climática (IVC) Predicho

Cali, Valle del Cauca — Transferencia Espacial del Modelo



(C) OpenStreetMap contributors (C) CARTO

NOTAS METODOLÓGICAS:
• Algoritmo Predictivo: LightGBM (Transferencia de Dominio). Universo analizado: 135 polígonos catastrales.
• Tratamiento de datos: Valores atípicos excluidos mediante rango intercuartílico (IQR). Proyección: Web Mercator (EPSG:3857).

✅ `Resultados de Cali, Valle del Cauca respaldados exitosamente en S3.`

In [ ]: